

Universidad
Rey Juan Carlos

Escuela Técnica Superior
de Ingeniería Informática

Grado en Ingeniería del Software

Curso 2025-2026

Trabajo Fin de Grado

DESPLIEGUE EN LA NUBE, AUTOMATIZACIÓN Y ESCALABILIDAD DE SPIRITBLADE

Autor: Jorge Andrés Echevarría
Tutor: Iván Chicano Capelo
Tutor: Francisco Gortázar Bellas



©2025 Jorge Andrés Echevarría
Algunos derechos reservados
Este documento se distribuye bajo la licencia
"Atribución-CompartirIgual 4.0 Internacional" de Creative Commons,
disponible en
https://creativecommons.org/licenses/by-sa/4.0/deed.es

Agradecimientos

Agradezco profundamente a mi familia por su incondicional aliento a lo largo de toda mi etapa académica, brindándome el apoyo necesario para culminar este proyecto.

Asimismo, quiero dar las gracias a mis compañeros y amigos, con quienes he compartido innumerables horas de estudio, resolución de problemas y aprendizaje mutuo a lo largo del grado, y cuyo compañerismo ha sido fundamental en este camino.

Resumen

Este Trabajo de Fin de Grado (TFG) aborda el despliegue en producción de la aplicación web *SPIRITBLADE* —una plataforma de análisis estadístico avanzado para *League of Legends* desarrollada en la primera fase de la titulación— aplicando metodologías modernas de *Site Reliability Engineering* (SRE, del inglés Ingeniería de Fiabilidad de Sitios) y arquitectura *Cloud-Native*.

El objetivo central del proyecto es transformar una aplicación de funcionamiento local en un sistema altamente disponible, resiliente y escalable de forma automática, operando estrictamente dentro de los límites de recursos de la capa gratuita de *Oracle Cloud Infrastructure* (OCI, del inglés). Para ello, la aplicación monolítica se ha empaquetado con *Docker*, garantizando la compatibilidad multi-arquitectura necesaria para ejecutarse en procesadores *Advanced RISC Machines* (ARM, del inglés). La provisión de la infraestructura (redes, bases de datos y nodos de cómputo) se ha automatizado mediante el paradigma de Infraestructura como Código (IaC, del inglés *Infrastructure as Code*) utilizando *Terraform*.

El núcleo orquestador del sistema recae en *Oracle Kubernetes Engine* (OKE, del inglés), donde se han implementado múltiples capas de autoescalado (*Horizontal Pod Autoscaler* o HPA, del inglés y *Cluster Autoscaler*) que permiten a la plataforma absorber picos masivos de tráfico mediante el provisionamiento dinámico de réplicas y nodos físicos. Adicionalmente, el ciclo de vida del *software* está respaldado por flujos de Integración y Entrega Continua (CI/CD, del inglés *Continuous Integration / Continuous Delivery*) automatizados en *GitHub Actions*, garantizando la calidad mediante análisis estático y un paso a producción ágil y desatendido.

Palabras clave:

- Computación en la Nube (Cloud Computing)
- DevOps / GitOps
- Orquestación de Contenedores (Kubernetes)
- Infraestructura como Código (Terraform)
- Integración y Entrega Continua (CI/CD)
- Escalabilidad

Índice de contenidos

Índice de figuras

Índice de códigos

1. Introducción y objetivos **1**

1.1. Contexto . . . 1
1.2. Trabajo previo . . . 2
1.3. Objetivos . . . 2
1.3.1. Objetivos funcionales . . . 2
1.3.2. Objetivos técnicos . . . 3

2. Metodología **5**

2.1. Planificación temporal . . . 6

3. Descripción funcional **8**

3.1. Descripción de la aplicación . . . 8
3.2. Resumen de módulos y perfiles de usuario . . . 9
3.2.1. Módulo de usuario no registrado (Anónimo) . . . 9
3.2.2. Módulo de usuario registrado . . . 10
3.2.3. Módulo de administración . . . 11
3.3. Vídeo demostrativo . . . 12
3.4. Referencias a anexos de funcionalidades y ejecución . . . 12

4. Descripción Informática **13**

4.1. Tecnologías . . . 14
4.2. Herramientas . . . 16
4.3. Arquitectura . . . 18
4.3.1. Arquitectura de despliegue (Cloud) . . . 18
4.4. Implementación . . . 20
4.4.1. Fase 1: Uso de caché con *Spring Cache* y *Redis* . . . 20
4.4.2. Fase 2: Despliegue en la nube (*Kubernetes*) . . . 23
4.4.3. Fase 3: Escalabilidad . . . 27

4.4.4. Fase 4: Despliegue continuo 30
4.5. Proceso de desarrollo 32
4.5.1. Integración, Entrega y Despliegue Continuo (*CI/CD*) 33
4.6. Pruebas de carga y validación de escalabilidad 37

5. Conclusiones y trabajos futuros **41**
5.1. Conclusiones 41
5.1.1. Reflexión sobre los objetivos funcionales 41
5.1.2. Reflexión sobre los objetivos técnicos 42
5.1.3. Conclusiones personales 43
5.2. Trabajos futuros 43

Bibliografía **45**

Anexos **47**

A. Instrucciones de Despliegue **49**
A.1. Requisitos previos 49
A.2. Fase 1: Provisión de Infraestructura (*Terraform*) 49
A.3. Fase 2: Configuración del Entorno *Kubernetes* 50
A.4. Fase 3: Entrega Continua (*CI/CD*) 51
A.5. Fase 4: Configuración de Dominio y DNS 51

Índice de figuras

2.1. Diagrama de Gantt ilustrando la planificación de fases. 7

3.1. Vista detallada del historial de partidas de un invocador. 9
3.2. Panel de control (*Dashboard*) de un usuario registrado. 10
3.3. Reporte táctico generado por la Inteligencia Artificial de Google Gemini. 11
3.4. Panel de administración con listado y filtrado de usuarios. 11

4.1. Diagrama global de la arquitectura de despliegue en *OCI*. 21
4.2. Diagrama topológico de la red virtual (*VCN*), sus subredes y puertas de enlace en Oracle Cloud. 24
4.3. Visión general de alto nivel de los recursos del clúster de Kubernetes. . . 26
4.4. Topología y escalado horizontal de la capa de Frontend en Kubernetes. . 29
4.5. Topología y escalado horizontal de la capa de Backend en Kubernetes. . . 30
4.6. Arquitectura de la capa de Datos (Persistencia externa y Caché de lectura). 31
4.7. Ejemplo de petición de integración (*Pull Request*) mostrando la convención de nomenclatura aplicada tanto al nombre de la rama superior como a los mensajes de los *commits* individuales subyacentes. 32
4.8. Captura de pantalla de los resultados del análisis estático de código en *SonarCloud*. 34
4.9. Diagrama de flujo de Integración Continua y validación de código. 34
4.10. Diagrama de flujo de Despliegue Continuo a Producción mediante versionado. 36
4.11. Evolución del autoescalado de *Pods* y Nodos en relación al porcentaje de uso de CPU durante la simulación de carga. 38

Índice de códigos

4.1. Uso de la anotación @Cacheable en el servicio de Riot Games para cachear datos de invocadores. 22
4.2. Fragmento de GitHub Actions ejecutando compilación nativa en matriz. . 35
4.3. Dockerfile de backend optimizado (sin compilación interna). 35

1 Introducción y objetivos

1.1. Contexto

El presente Trabajo de Fin de Grado constituye la continuación directa y secuela del anterior Trabajo de Fin de Grado asociado a la plataforma *SPIRITBLADE*, desarrollado como culminación de la otra titulación cursada. Mientras que el primer Trabajo de Fin de Grado resolvió el desafío de concebir y programar el sistema, el producto resultante, si bien funcional en entornos locales de desarrollo, carecía de un entorno de producción real que garantizara su disponibilidad a través de Internet.

Es en esta coyuntura donde nace la necesidad de este segundo trabajo, cuyo enfoque pivota radicalmente hacia la *Site Reliability Engineering* (*SRE* del inglés), las metodologías *DevOps* y la arquitectura *Cloud-Native*. El contexto actual de la industria del *software* demanda que las aplicaciones sean resilientes ante fallos, capaces de escalar dinámicamente para absorber picos repentinos de tráfico, y que su ciclo de entrega esté completamente automatizado mediante *Continuous Integration / Continuous Deployment* (*CI/CD* del inglés).

Por tanto, la justificación de este proyecto radica en la imperativa necesidad de desplegar *SPIRITBLADE* en una infraestructura en la nube pública, abstrayendo la complejidad subyacente mediante el uso de contenedores (*Docker*), orquestando los servicios con el estándar de la industria (*Kubernetes*), integrando la base de datos como pieza fundamental para asegurar la persistencia y disponibilidad de la información, y gestionando todo el ciclo de vida de la infraestructura a través de *Infraestructura como Código* (*IaC* del inglés).

1.2. Trabajo previo

Como punto de partida, se ha heredado la base tecnológica (*codebase*) del proyecto anterior. Dicho sistema original consistía en una aplicación funcional compuesta por un cliente web tipo *Single Page Application* (*SPA* del inglés) desarrollado con *Angular*, un servidor *backend* implementado con *Spring Boot* en *Java* y una base de datos relacional MySQL para la persistencia de los datos.

El servidor centraliza la lógica de negocio, comunicándose con las *Application Programming Interface* (*APIs* del inglés) oficiales de Riot Games para extraer los datos de las partidas y transformarlos en información estadística, así como con el motor de Inteligencia Artificial de Google Gemini para la generación de reportes tácticos personalizados. La persistencia de datos se aseguraba mediante MySQL y los accesos estaban protegidos con autenticación basada en *JSON Web Tokens* (*JWT* del inglés).

El TFG anterior resolvió con éxito los retos de diseño de interfaz, programación de los servicios y estructuración del modelo de datos, consiguiendo un producto funcional capaz de ejecutarse en entornos de desarrollo locales. Este trabajo previo constituye el código base sobre el cual se aplicarán las nuevas metodologías SRE, de orquestación y de despliegue continuo que conforman el objeto central de la presente memoria.

1.3. Objetivos

El objetivo principal de este trabajo es lograr el paso a producción de la aplicación web *SPIRITBLADE*, garantizando un entorno escalable, seguro y gestionado de manera automática que abstraiga la complejidad operacional. Para estructurar este propósito general, se han definido una serie de objetivos específicos divididos en funcionales y técnicos.

1.3.1. Objetivos funcionales

Los objetivos funcionales reflejan las metas observables desde la perspectiva de la disponibilidad y el servicio ofrecido a los usuarios de la plataforma:

- **Disponibilidad pública ininterrumpida:** Desplegar la aplicación en un dominio público (`spiritblade.dev`), adquirido mediante el registrador *name.com*, garantizando su acceso continuo desde cualquier parte del mundo.

- **Tolerancia a fallos:** Garantizar que la caída de un servidor o componente no implique la interrupción del servicio global, implementando mecanismos de recuperación automática.
- **Escalabilidad bajo demanda:** Conseguir que la plataforma sea capaz de ajustar dinámicamente sus recursos para absorber incrementos repentinos de tráfico de usuarios sin degradar la experiencia de uso, y reducirse en períodos valle para no malgastar recursos.
- **Seguridad de las comunicaciones:** Proveer cifrado de extremo a extremo en todas las interacciones cliente-servidor mediante la gestión y renovación automática de certificados *Secure Sockets Layer / Transport Layer Security* (*SSL/TLS* del inglés) para el protocolo *Hypertext Transfer Protocol Secure* (*HTTPS* del inglés).
- **Coste operativo cero:** Lograr que toda la infraestructura, bases de datos y balanceadores operen estrictamente dentro de los límites de las capas gratuitas, en este caso, el *Always Free Tier* de *Oracle Cloud Infrastructure* (*OCI* del inglés).

1.3.2. Objetivos técnicos

Los objetivos técnicos abordan las prácticas de ingeniería, metodologías y herramientas que habilitan el cumplimiento de los objetivos funcionales:

- **Empaquetado en contenedores:** Empaquetar tanto el cliente web como el servidor *backend* en imágenes de contenedor estándar (*Docker*), garantizando que se ejecuten idénticamente en cualquier entorno.
- **Soporte Multi-Arquitectura:** Adaptar los procesos de compilación para que las imágenes generadas sean compatibles nativamente con los procesadores ARM de la nube de Oracle, evadiendo los tradicionales cuellos de botella de emulación.
- **Infraestructura como Código (*IaC*):** Definir y aprovisionar toda la infraestructura en la nube (redes, nodos, bases de datos, almacenamiento) mediante código declarativo utilizando *Terraform*, eliminando la configuración manual.
- **Orquestación avanzada:** Desplegar un clúster de *OKE* como panel de control central para la gestión del ciclo de vida de los contenedores.
- **Despliegue de Autoescaladores:** Configurar un *HPA* para replicar los servicios en base a métricas de *CPU* y un *Cluster Autoscaler* para aprovisionar nuevas máquinas físicas en *OCI* cuando los recursos del clúster se agoten.
- **Integración y Entrega Continua (*CI/CD*):** Implementar *pipelines* automatizados con *GitHub Actions* que compilen, analicen (mediante *SonarCloud*) y des-

plieguen nuevas versiones de la aplicación sin intervención humana tras la integración de nuevo código en el repositorio principal.

2

Metodología

Este capítulo describe el enfoque de trabajo empleado para la realización de la segunda iteración del proyecto, detallando las fases del ciclo de vida de la infraestructura y el despliegue continuo, así como la planificación temporal a través de un diagrama de Gantt.

El desarrollo de este segundo TFG se ha estructurado siguiendo un proceso iterativo e incremental en espiral, planificado mediante reuniones de seguimiento periódicas con el tutor para definir y ajustar el ámbito del proyecto. En lugar de centrarse en la construcción de nueva lógica de negocio, esta metodología se ha orientado hacia la provisión de infraestructura inmutable, el despliegue en la nube y la escalabilidad, validando cada incremento antes de pasar a la siguiente fase de complejidad arquitectónica. A continuación, se detallan las fases en las que se ha dividido el trabajo, indicando los aspectos técnicos de alto nivel abordados en cada una:

- **Fase 1: Uso de caché con Spring Cache y Redis.** Integración de un sistema de caché distribuida en memoria (Redis) en el *backend* para reducir la latencia de las respuestas y disminuir la carga sobre las API externas, optimizando el rendimiento antes de su paso a la nube.
- **Fase 2: Despliegue en la nube (*Kubernetes*).** Empaquetado de todos los componentes de la aplicación y creación de la infraestructura base en un proveedor

de nube pública (Oracle Cloud) mediante *IaC*. Esta fase culmina con el despliegue funcional en un clúster de *Kubernetes* administrado mediante *Oracle Kubernetes Engine* (*OKE* del inglés).

- **Fase 3: Escalabilidad.** Configuración y prueba de mecanismos de elasticidad para absorber dinámicamente variaciones en el tráfico, implementando autoescalado tanto a nivel de *pods* (*Horizontal Pod Autoscaler* o *HPA* del inglés) como a nivel de infraestructura física del clúster de *Kubernetes*.

- **Fase 4: Despliegue continuo.** Automatización completa del flujo de entrega (*CI/CD*) empleando *GitHub Actions*, garantizando que cualquier cambio en el código o en la configuración se pruebe y despliegue en los entornos de nube sin intervención manual.

- **Fase 5: Redacción de memorias.** Redacción conjunta y en paralelo de la presente memoria y la del primer proyecto, documentando en detalle las arquitecturas *Cloud-Native*, el escalado y los flujos de puesta en producción.

- **Fase 6: Preparación y defensa.** Preparación del material de apoyo visual, las demostraciones en vivo de autoescalado y el discurso para la exposición conjunta en las semanas previas a la defensa ante el tribunal evaluador.

2.1. Planificación temporal

Para ilustrar la planificación y ejecución de las fases descritas, se ha elaborado un cronograma con las fechas de inicio y fin estimadas para cada etapa principal. En la Tabla 2.1 se presenta el resumen de fechas:

Fase	**Inicio**	**Fin**
Fase 1: Uso de caché	01/01/2026	15/01/2026
Fase 2: Despliegue en la nube	16/01/2026	30/01/2026
Fase 3: Escalabilidad	31/01/2026	28/02/2026
Fase 4: Despliegue continuo	01/03/2026	15/03/2026
Fase 5: Redacción de memorias	16/03/2026	25/05/2026
Fase 6: Preparación y defensa	26/05/2026	09/06/2026

Tabla 2.1: Cronograma de hitos del proyecto.

De forma complementaria, la evolución temporal de las fases descritas anteriormente se ilustra en el diagrama de Gantt mostrado en la Figura 2.1.

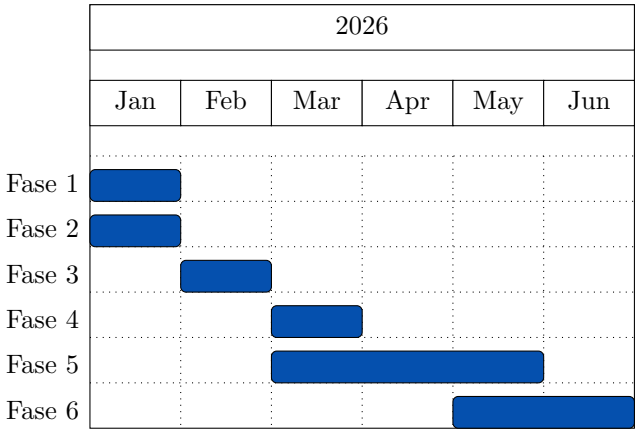


Figura 2.1: Diagrama de Gantt ilustrando la planificación de fases.

3

Descripción funcional

Este capítulo proporciona un resumen de la funcionalidad previa desarrollada en el TFG anterior, la cual constituye la base de la plataforma *SPIRITBLADE*. Aunque las funcionalidades en sí mismas fueron creadas durante el primer TFG, se describen aquí brevemente para aportar contexto sobre el sistema que el presente proyecto se encarga de optimizar, escalar y desplegar en producción.

3.1. Descripción de la aplicación

SPIRITBLADE es una plataforma web orientada al análisis estadístico avanzado para el videojuego competitivo *League of Legends*. Su propósito principal es transformar los datos en bruto proporcionados por la *API* oficial de *Riot Games* en información estructurada, digerible y útil para los jugadores, potenciando este análisis mediante Inteligencia Artificial (*IA*).

La plataforma permite a los usuarios buscar perfiles de invocadores (*summoners*), inspeccionar historiales de partidas recientes con un gran nivel de detalle (rendimiento, puntuación de visión individual, daño infligido, duración, *KDA*), y solicitar a un motor de *IA* (*Google Gemini*) un informe exhaustivo que destaca errores tácticos y proporciona pautas de mejora basadas en el rendimiento empírico del jugador en sus últimas

competiciones. Todo el contenido dinámico e imágenes estáticas (como campeones o posiciones) se extraen e integran haciendo uso de la *Content Delivery Network* (*CDN* del inglés) *Data Dragon* de *Riot*.

3.2. Resumen de módulos y perfiles de usuario

Las capacidades del sistema están segmentadas en tres niveles principales de acceso, proporcionando una experiencia progresiva según el grado de implicación del usuario con la plataforma:

3.2.1. Módulo de usuario no registrado (Anónimo)

El usuario anónimo tiene acceso a las funciones públicas de consulta ágil. Puede buscar invocadores por su *Riot ID* (`gameName#tagLine`), visualizar su rango competitivo (*Elo* o nivel de *Tier/División/LP*), consultar estadísticas sobre el *top 3* de sus campeones más jugados (maestrías) e inspeccionar un historial completo y paginable de sus partidas recientes. Este historial, ilustrado en la Figura 3.1, incluye métricas avanzadas como el *Vision Score* (puntuación de visión) por participante, duración del encuentro, resultado y rol desempeñado.

Asimismo, la página de inicio ofrece acceso rápido mostrando una lista con las últimas búsquedas realizadas globalmente en la plataforma y estadísticas agregadas (tasa de victorias media por campeón o *KDA* medio por rol), apoyadas por un sistema de caché de alta velocidad (Redis) que evita la sobrecarga de la API externa.

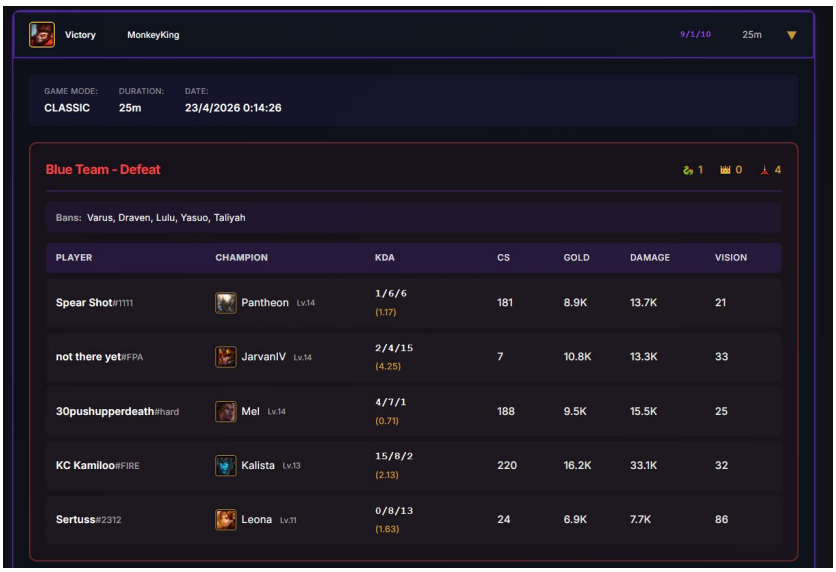


Figura 3.1: Vista detallada del historial de partidas de un invocador.

3.2.2. Módulo de usuario registrado

Los usuarios que se registran e inician sesión mediante autenticación *JWT* acceden a un panel de control personalizado (*Dashboard*), como se ilustra en la Figura 3.2. Este perfil permite vincular una cuenta oficial de *League of Legends* al usuario del sistema, habilitando el seguimiento automático de estadísticas, la evolución gráfica del rango competitivo a lo largo del tiempo (diferenciando colas como *SoloQ* y *Flex*), y la gestión de una lista de invocadores favoritos para su acceso rápido.

La característica más destacada de este módulo es la capacidad de invocar el motor de *IA* para generar reportes tácticos personalizados, tal y como se muestra en la Figura 3.3. Además, pueden gestionar su propia foto de perfil (alojada en repositorios de objetos MinIO) y editar su información personal.

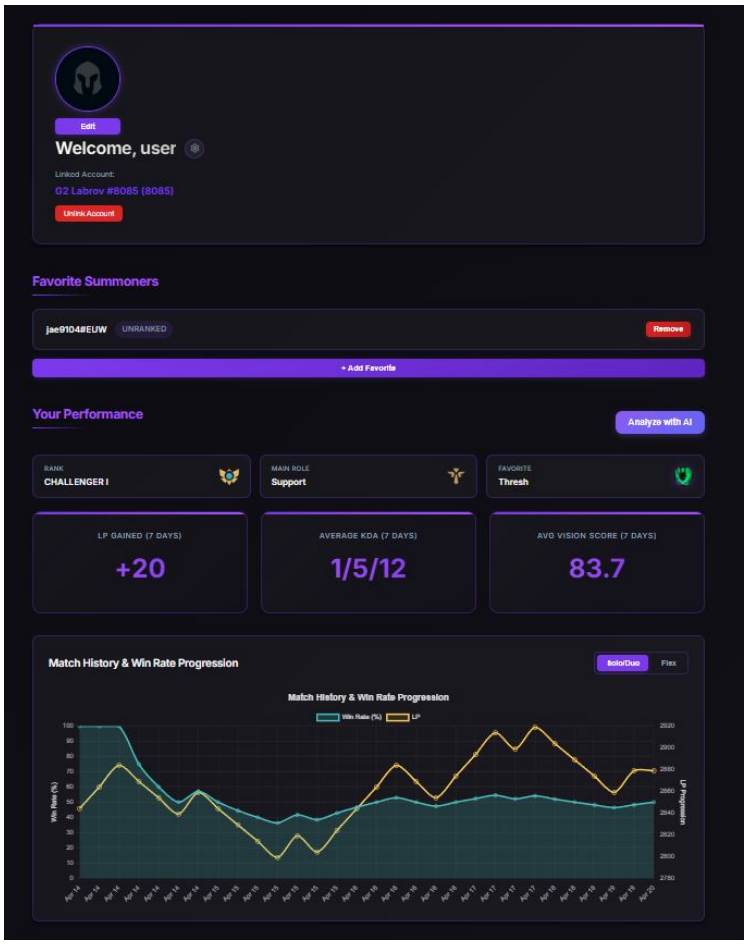


Figura 3.2: Panel de control (*Dashboard*) de un usuario registrado.

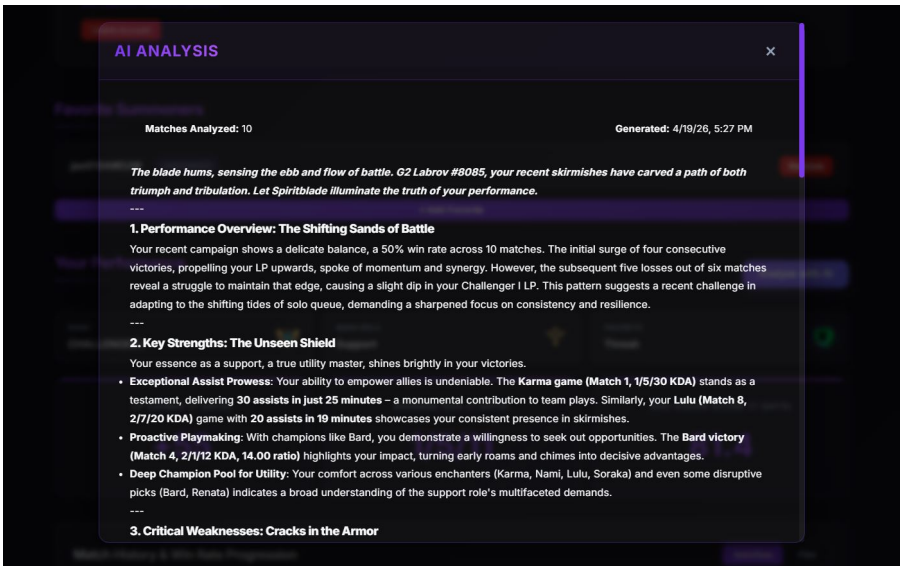


Figura 3.3: Reporte táctico generado por la Inteligencia Artificial de Google Gemini.

3.2.3. Módulo de administración

Reservado para los gestores de la plataforma. Incluye un panel de control interactivo con capacidades de búsqueda avanzada (filtros por nombre, email, rol o estado activo) y paginación para listar la totalidad de los usuarios, como se observa en la Figura 3.4. Permite, de manera dinámica y sin recargas, habilitar, deshabilitar o eliminar cuentas de usuario. También incorpora la modificación de roles (ascender a otros usuarios a administradores) para garantizar la correcta moderación y mantenimiento del servicio.

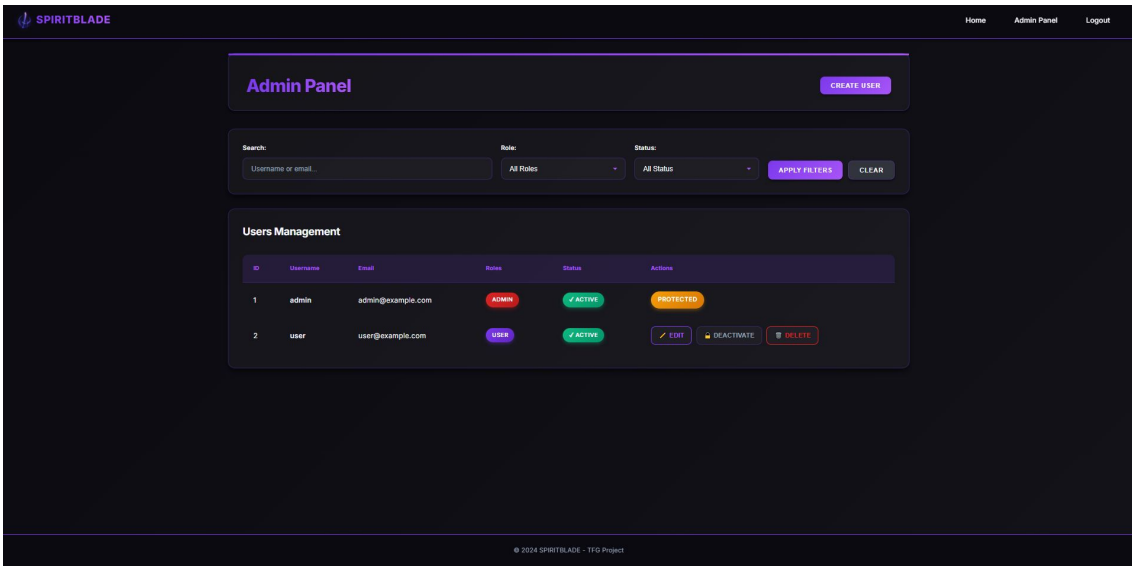


Figura 3.4: Panel de administración con listado y filtrado de usuarios.

3.3. Vídeo demostrativo

Para facilitar la comprensión del funcionamiento de la interfaz web, la fluidez de la navegación entre los distintos módulos y la visualización de las métricas obtenidas en tiempo real, se ha grabado una demostración de uso de la aplicación.

El vídeo, que cuenta con una explicación guiada detallando las acciones realizadas en la plataforma, está disponible en línea para su visualización en el enlace .

`https://youtu.be/JmKdQhNpecU`

3.4. Referencias a anexos de funcionalidades y ejecución

Debido a que el grueso del diseño lógico e interfaces de usuario corresponde al TFG anterior, esta memoria se centra en la arquitectura *Cloud-Native*. Dado que la plataforma está diseñada para ser escalable y orquestada en la nube mediante tecnologías como *Docker* y *Kubernetes*, las **instrucciones detalladas para ejecutar la aplicación y desplegar la infraestructura** en producción se han incluido en el **Anexo A: Instrucciones de Despliegue** al final de este documento.

4

Descripción Informática

El código fuente completo de la aplicación, así como los manifiestos de infraestructura y orquestación (*Terraform* y *Kubernetes*), se encuentran alojados y versionados en un repositorio público de *GitHub* (`https://github.com/codeurjc-students/2025-SPIRITBLADE`).

La arquitectura de la aplicación original se rige por el patrón *SPA*. En este modelo, el cliente web (desarrollado en *Angular*) carga un único documento *HTML* y reescribe dinámicamente el contenido del mismo en respuesta a las acciones del usuario, comunicándose de manera asíncrona con un servidor o *backend* remoto (desarrollado en *Spring Boot*) a través de una *API RESTful*. La base de datos relacional (MySQL) actúa como capa de persistencia definitiva, apoyada por una caché en memoria distribuida (Redis) para reducir latencias en lecturas repetitivas.

A continuación, se resumen los componentes clave de la infraestructura de despliegue en formato de lista:

- **Tipo:** Aplicación Web *SPA*, *API REST*, Microservicios *Cloud-Native*.
- **Tecnologías base:** *Java* (*Spring Boot*), *TypeScript* (*Angular*), *MySQL*, *Redis*.
- **Tecnologías Cloud e *IaC*:** *OCI*, *Terraform*.

- **Orquestación y Contenedores:** *Docker*, *Kubernetes* (*OKE*), *Helm*, *NGINX Ingress*.
- **Control de calidad:** *SonarCloud* (Análisis estático), *Artillery* (Pruebas de carga).
- **Proceso de desarrollo:** Metodología Iterativa, *DevOps* (*GitOps*), *CI/CD*.

4.1. Tecnologías

Aparte de las tecnologías núcleo de desarrollo (*Spring Boot* y *Angular*) documentadas extensamente en la primera memoria, para lograr el despliegue en la nube y la escalabilidad del sistema se ha apoyado en las siguientes plataformas:

Oracle Cloud Infrastructure (OCI)

Es la plataforma de computación en la nube pública de Oracle, elegida como el proveedor de infraestructura base para este proyecto [1]. OCI no proporciona acceso directo a servidores físicos (*bare metal*) en su capa gratuita, sino que ofrece instancias de máquinas virtuales completamente funcionales y aisladas dentro de una red virtual en la nube (*VCN* del inglés), sobre las cuales el usuario retiene control total a nivel de sistema operativo.

Si bien OCI es compatible con la provisión de instancias tradicionales de arquitectura x86 (tales como AMD o Intel), para este despliegue se ha optado estratégicamente por utilizar procesadores basados en la arquitectura *Advanced RISC Machines* (ARM, concretamente instancias *Ampere A1*). Esta decisión técnica se justifica fundamentalmente por el generoso nivel gratuito permanente (*Always Free Tier*) que ofrece Oracle en exclusiva para esta arquitectura.

Este modelo gratuito impone restricciones máximas compartidas para toda la cuenta, que dictaminarán la filosofía de diseño y escalado del resto del proyecto:

- **Cómputo y Memoria:** Un máximo estricto e inamovible de 4 núcleos físicos (*OCPUs*) ARM y 24 GB de memoria RAM en total.
- **Almacenamiento en bloque:** Un límite de 200 GB totales para discos duros virtuales (*Block Volumes*), capacidad que se agota velozmente al aprovisionar los discos de arranque obligatorios para múltiples nodos.

A pesar de estas cotas, los recursos resultan holgados para sostener y orquestar un clúster de Kubernetes en producción eficiente y de alto rendimiento.

Docker

Se ha utilizado *Docker* [2] como la plataforma estándar para la contenerización. *Docker* permite empaquetar de forma inmutable las aplicaciones (tanto el cliente *Angular* como el servidor *Spring Boot*) junto con todas sus dependencias, librerías y entornos de ejecución subyacentes en unidades estandarizadas denominadas contenedores. Este paradigma garantiza una ejecución consistente y predecible de los servicios, eliminando las discrepancias de comportamiento independientemente de si el código se ejecuta en un entorno de desarrollo local o en los nodos distribuidos de producción en la nube.

Kubernetes (OKE)

Kubernetes [3] es el sistema de orquestación de contenedores *open-source* consolidado como el estándar de la industria. Su propósito es automatizar el despliegue masivo, la gestión, el balanceo de carga y el ajuste de escala de aplicaciones contenerizadas. Para este proyecto, en lugar de instalar y operar un clúster de *Kubernetes* en crudo, se ha utilizado *Oracle Kubernetes Engine* (OKE), su variante gestionada dentro de OCI. OKE delega la carga administrativa y el mantenimiento crítico del plano de control maestro (*Control Plane*) directamente a Oracle, permitiendo al equipo de desarrollo centrarse de forma exclusiva en la configuración de los nodos de trabajo (*Worker Nodes*) y en la resiliencia de las cargas de trabajo desplegadas.

Terraform

Desarrollada por HashiCorp, *Terraform* es la herramienta principal utilizada en el proyecto para aplicar el paradigma de Infraestructura como Código (*IaC* del inglés). Permite definir, aprovisionar y gestionar toda la topología de la nube de forma totalmente declarativa mediante archivos de configuración *HCL*.

Una de las ventajas más sustanciales de *Terraform* radica en ofrecer un ecosistema agnóstico respecto a procesos (*workflow-agnostic*) frente a cualquier proveedor de nube. Esto implica que sus comandos principales de ciclo de vida (*init*, *plan*, *apply* y *destroy*) y la estructura organizativa de su lenguaje HCL se mantienen inmutables, independientemente de si la infraestructura reside en OCI, *Amazon Web Services* (AWS) o *Azure*. Sin embargo, es imperativo establecer una distinción arquitectónica crítica: **el código de definición de recursos no es directamente portable entre nubes**. Dado que

cada proveedor encapsula tecnologías y denominaciones propietarias (por ejemplo, definiendo un servidor *computacional* bajo el recurso `oci_core_instance` en Oracle frente a `aws_instance` en AWS), una migración de proveedor obligaría a reescribir integralmente la capa de infraestructura (*IaC*). No obstante, este enfoque arquitectónico reduce la curva de aprendizaje, facilita la trazabilidad histórica y auditoría mediante control de versiones en *Git*, y asegura que el despliegue del entorno en el proveedor elegido se ejecute de forma determinista y auditable.

Helm

Helm [4] actúa como el gestor de paquetes de *software* por excelencia para el ecosistema *Kubernetes*. Facilita enormemente la definición, instalación y actualización de aplicaciones complejas, las cuales se empaquetan en colecciones estructuradas de manifiestos y plantillas denominadas *Charts*. En el presente proyecto, *Helm* se ha empleado fundamentalmente para automatizar la intrincada puesta en marcha del motor *Redis* en modo clúster (*Primary-Replica*), abstrayendo al desarrollador de la ardua tarea de crear y enlazar manualmente docenas de servicios, despliegues y volúmenes de persistencia.

4.2. Herramientas

Las herramientas utilizadas por el equipo de desarrollo para la programación, pruebas y la administración interactiva de la infraestructura desplegada se resumen a continuación:

GitHub Actions

Es una plataforma de Integración y Entrega Continua (*CI/CD*) fuertemente integrada en el ecosistema del repositorio de *GitHub*. Permite automatizar los flujos de trabajo (*workflows*) ejecutando secuencias de comandos en servidores virtuales hospedados directamente por *Microsoft*. En este proyecto, se erige como el núcleo orquestador del ciclo de vida del *software*, encargándose de compilar las imágenes *Docker*, ejecutar análisis estáticos y pruebas, y actualizar los recursos en el clúster de producción de forma totalmente desatendida tras cada integración de código.

OCI CLI

La Interfaz de Línea de Comandos de OCI (*CLI* del inglés) oficial de *Oracle* resulta esencial para administrar los recursos en la nube desde terminales automatizadas, sin requerir clics ni accesos manuales a la consola web. En los *pipelines* de entrega continua de *GitHub Actions*, esta herramienta asume el rol crítico de generar credenciales dinámicas de un solo uso que permiten la conexión segura e imperativa a la *API* del clúster de *Kubernetes* [5].

kubectl

Es la herramienta de línea de comandos maestra y el principal mecanismo de interacción con *Kubernetes*. Actúa como puente de comunicación directo entre el administrador humano (o un sistema automatizado) y el plano de control (*Control Plane*) del clúster. Se ha empleado exhaustivamente durante las fases de validación para inyectar manifiestos de configuración, inspeccionar el estado en tiempo real de los contenedores (*Pods*), visualizar registros de ejecución (*logs* del inglés) y ejecutar tareas de diagnóstico directamente contra los entornos de producción [6].

Artillery

Es una herramienta moderna de código abierto, escrita en *Node.js*, diseñada específicamente para la ejecución de pruebas dinámicas de carga y estrés. Su uso resulta fundamental para la disciplina de ingeniería de confiabilidad de sitios (*SRE* del inglés). *Artillery* se ha utilizado para simular avalanchas de tráfico virtual (inyectando miles de peticiones *HTTP* por segundo) sobre los *endpoints* críticos de la plataforma. De esta manera, se ha podido certificar de forma empírica que los mecanismos de autoescalado (*HPA* y *Cluster Autoscaler*) reaccionan y aprovisionan recursos correctamente antes de exponer el servicio al público real [7].

SonarCloud

Constituye una plataforma analítica en la nube (*Software as a Service*) orientada a la inspección continua de la calidad del código fuente. Completamente integrada en los flujos de *GitHub Actions*, *SonarCloud* realiza escaneos profundos de seguridad estática (*SAST* del inglés). Su labor principal es descubrir olores de código (*code smells*), detectar niveles inaceptables de código duplicado e impedir la inyección de vulnerabilidades conocidas (*Bugs* y *Security Hotspots*) en la rama principal, asegurando así que

las modificaciones de los desarrolladores mantengan un alto estándar de calidad.

4.3. Arquitectura

A diferencia de la memoria anterior centrada en los componentes lógicos de *software*, la arquitectura descrita en este documento ilustra las capas de despliegue y la topología física y lógica *Cloud-Native*. Dado que el modelo de dominio, la arquitectura del cliente web y la estructura interna del servidor no han sufrido cambios estructurales relevantes, este apartado omitirá dichos detalles para centrarse de forma exclusiva en la orquestación, las redes y los servicios en la nube.

4.3.1. Arquitectura de despliegue (Cloud)

El diseño del entorno de producción se ha materializado combinando recursos delegados al proveedor de la nube (*Oracle Cloud Infrastructure* o OCI) y recursos orquestados internamente dentro de un clúster de *Kubernetes*. Para facilitar la comprensión a perfiles familiarizados con otros proveedores líderes del mercado como *Amazon Web Services* (AWS) o *Microsoft Azure*, se desglosará la arquitectura separando claramente qué componentes son nativos del proveedor y cuáles son administrados de forma interna por el proyecto.

Servicios Gestionados por el Proveedor (OCI)

Un "servicio gestionado" (*Managed Service*) es aquel en el que el proveedor de la nube (en este caso, Oracle) asume la responsabilidad total sobre la infraestructura subyacente, el mantenimiento del sistema operativo, la aplicación de parches de seguridad y, a menudo, la disponibilidad y las copias de seguridad. El equipo de desarrollo simplemente consume el servicio a través de su interfaz o *API*, sin tener acceso ni control sobre los servidores físicos que lo ejecutan. En la arquitectura de este proyecto, los siguientes servicios entran en esta categoría:

- **Virtual Cloud Network (VCN):** Constituye la red privada virtual fundacional del proyecto en OCI (el equivalente directo a *Amazon VPC* en AWS o *Virtual Network* en Azure). Define el espacio de direcciones IP y aísla lógicamente los recursos. Se ha segmentado estratégicamente en subredes públicas (con acceso directo a Internet a través de un *Internet Gateway*) y subredes privadas (donde residen los servidores de aplicaciones para evitar su exposición directa).

- **Oracle Kubernetes Engine (OKE):** Es el servicio gestionado de *Kubernetes* nativo de Oracle (equivalente a *EKS* en AWS o *AKS* en Azure). En este modelo, Oracle provisiona, asegura y mantiene gratuitamente el Plano de Control (*Control Plane*) del clúster (el cerebro que toma las decisiones de orquestación), liberando al equipo de su compleja administración. El proyecto únicamente provisiona y administra (mediante la capa gratuita) los Nodos de Trabajo (*Worker Nodes*) físicos que ejecutarán los contenedores.

- **Autonomous Database (ADB):** Es el servicio de base de datos relacional nativo y completamente gestionado de Oracle, que actúa como el equivalente a *Amazon RDS*. Sustituye al contenedor de *MySQL* utilizado en los entornos de desarrollo locales. Al externalizar la persistencia a *ADB* mediante conexiones cifradas (TCPS), se libera a los nodos de *Kubernetes* de la pesada carga de procesar transacciones en disco, delegando las copias de seguridad y la alta disponibilidad en la infraestructura autónoma de Oracle.

- **OCI Object Storage:** Es el servicio de almacenamiento de objetos estáticos en la nube propietario de Oracle, equivalente a *Amazon S3* o *Azure Blob Storage*. Es importante aclarar que **no se trata de MinIO**. Mientras que *MinIO* es una herramienta de código abierto que emula el comportamiento de *S3* y que fue utilizada en la primera fase del proyecto para el desarrollo local, *OCI Object Storage* es la solución comercial, nativa y gestionada del proveedor, que ofrece durabilidad de datos y alta disponibilidad a escala global sin consumir recursos de nuestro clúster.

Componentes Orquestados dentro del Clúster (Kubernetes)

A diferencia de los servicios gestionados mencionados anteriormente, los recursos descritos en esta sección se despliegan, configuran y ejecutan estrictamente dentro del ecosistema de *Kubernetes* (dentro de los Nodos de Trabajo de la *VCN*). Su ciclo de vida, disponibilidad y escalabilidad son responsabilidad directa de las configuraciones, manifiestos y políticas definidas en el proyecto, operando de manera independiente a los servicios gestionados de Oracle:

- **Controlador de *Ingress* (NGINX *Ingress Controller*):** Mientras que un proveedor suele ofrecer balanceadores de carga como un servicio gestionado externo, en esta arquitectura el enrutamiento inteligente (Capa 7 del modelo OSI) se realiza internamente. El controlador NGINX se despliega como un conjunto de contenedores (*Pods*) dentro de *Kubernetes*. Su función es interceptar el tráfico HTTP/HTTPS entrante, gestionar la terminación de los certificados de seguridad

TLS (apoyándose en el gestor interno *cert-manager*) y aplicar reglas de enrutamiento basadas en la ruta (*path*) para dirigir las peticiones hacia el cliente web o la *API* del servidor.

- **Aplicaciones sin estado (*Stateless*):** Los módulos lógicos desarrollados para este proyecto —el cliente web (*Frontend* en Angular) y el servidor (*Backend* en Spring Boot)— se despliegan como aplicaciones *stateless*. Al no almacenar datos permanentes en sus discos locales, *Kubernetes* tiene libertad absoluta para destruir réplicas defectuosas o crear nuevas de forma instantánea para equilibrar la carga (*autoscaling*) sin que se produzca pérdida de información o interrupción del servicio al usuario final.

- **Caché con estado (*Stateful* - Redis):** Constituye la única pieza con estado persistente alojada internamente en el clúster. Se despliega mediante el gestor de paquetes *Helm* y actúa como un intermediario de lectura extremadamente rápido. Su presencia evita consultas redundantes masivas a la base de datos externa (*ADB*) y a la *API* externa de Riot Games. Dado que opera en memoria RAM dentro de los propios nodos de *Kubernetes*, su gestión, persistencia temporal y tolerancia a fallos dependen íntegramente de la configuración de orquestación interna (arquitectura Primario-Réplica) y no de un servicio gestionado de Oracle.

Para ilustrar de forma concisa esta topología física y lógica, en la Figura 4.1 se expone el diagrama completo de la infraestructura en la nube de *Oracle*.

4.4. Implementación

La implementación de la arquitectura y la infraestructura se ha llevado a cabo de manera iterativa, atravesando fases bien definidas para evolucionar desde un entorno monolítico local hacia un despliegue en la nube elástico y automatizado.

4.4.1. Fase 1: Uso de caché con *Spring Cache* y *Redis*

Objetivos de la fase

El objetivo de esta iteración fue optimizar el rendimiento del servidor y prevenir bloqueos por exceso de peticiones (*rate limits* del inglés) en la *API* externa de *Riot Games*, mediante la implementación de un sistema de caché distribuida.

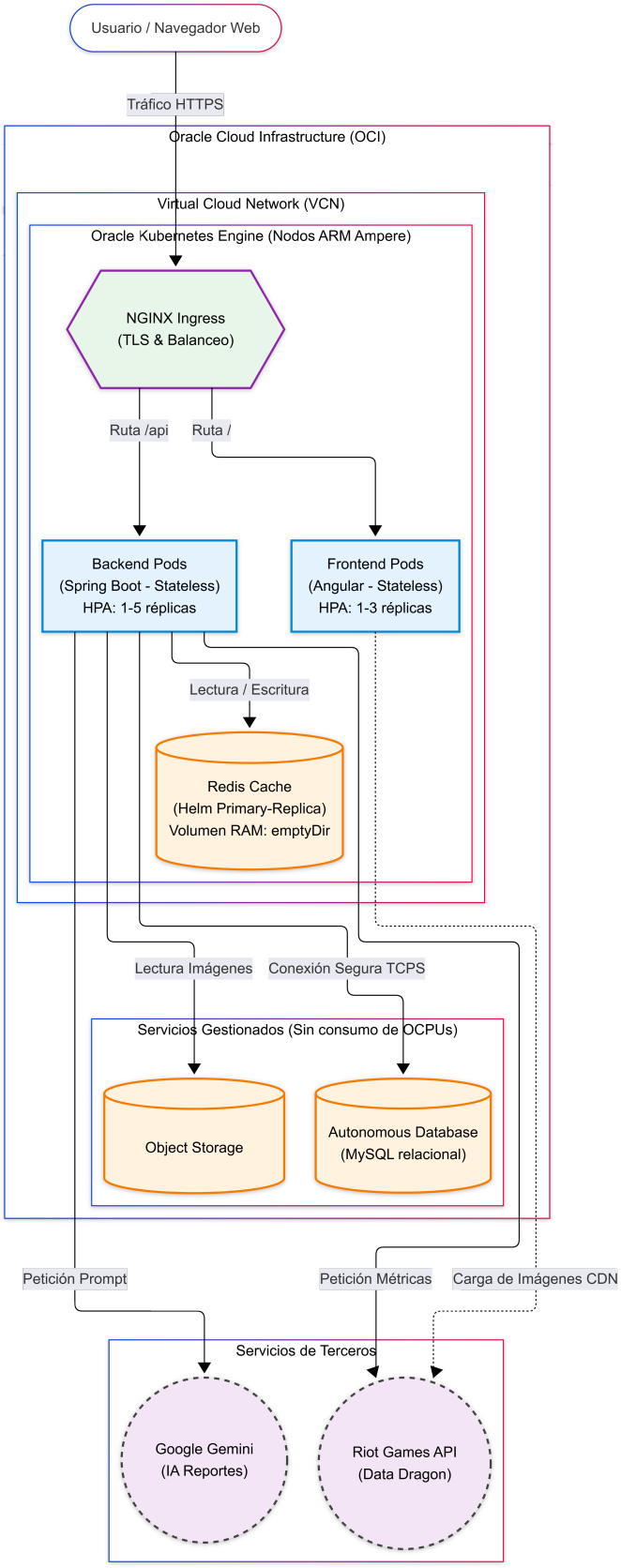


Figura 4.1: Diagrama global de la arquitectura de despliegue en *OCI*.

Detalles de implementación

Se integró *Redis* como almacén de clave-valor en memoria. A nivel de *software*, se empleó *Spring Cache* anotando los métodos críticos (`@Cacheable`). Se configuraron múltiples tiempos de vida (*TTL* del inglés) optimizados: 24 horas para recursos estáticos inmutables (campeones, historial de partidas finalizadas), 1 hora para maestrías y 10 minutos para datos altamente dinámicos (rango e invocadores).

Para ilustrar esta integración, en el Código 4.1 se expone un fragmento representativo del servicio encargado de obtener los datos de un invocador.

Código 4.1: Uso de la anotación @Cacheable en el servicio de Riot Games para cachear datos de invocadores.

```
1  @Cacheable(value = "summoners", key = "#riotId")
2  public SummonerDTO getSummonerByName(String riotId) {
3      // Logica de extraccion de datos
4      String[] parts = parseRiotId(riotId);
5      String gameName = parts[0];
6      String tagLine = parts[1];
7
8      String puuid = fetchPuuidByRiotId(riotId, gameName, tagLine);
9      RiotSummonerDTO riotSummoner = fetchSummonerByPuuid(riotId, puuid);
10     RiotLeagueEntryDTO rankedEntry = fetchRankedEntry(puuid);
11
12     return createSummonerDTO(riotSummoner, rankedEntry);
13 }
```

Mediante este enfoque declarativo, la ejecución del método es interceptada automáticamente. Al recibir una petición, *Spring* verifica en primer lugar si en la memoria de *Redis* existe un registro bajo el espacio `summoners` asociado a la clave correspondiente (el `riotId` del usuario). Si los datos están presentes y no han expirado, se devuelve la respuesta instantáneamente eludiendo la ejecución del código. Únicamente en caso de no encontrarlos (*cache miss*), se ejecuta la lógica secuencial que realiza las peticiones a la *API* externa, guardando automáticamente el resultado devuelto en la caché antes de finalizar.

Problemas encontrados y soluciones

El reto principal fue la correcta serialización de tipos polimórficos de *Java* a texto plano en *Redis*. La solución requirió configurar un `GenericJackson2JsonRedisSerializer` con un `ObjectMapper` personalizado que adjunta metadatos de tipo, garantizando la reconstrucción estructural íntegra tras un acierto en caché (*cache hit*).

4.4.2. Fase 2: Despliegue en la nube (*Kubernetes*)

Objetivos de la fase

Contenerizar todos los módulos de la aplicación y provisionar la infraestructura subyacente en la nube mediante código, posibilitando la ejecución en un clúster gestionado y eludiendo las limitaciones de la capa gratuita de *Oracle Cloud* (*Always Free Tier*).

Detalles de implementación

La provisión de la nube se programó íntegramente de forma declarativa utilizando módulos de *Terraform*. En primer lugar, se diseñó una red virtual (*VCN*) fuertemente segmentada para garantizar la seguridad perimetral. El diseño de red divide la infraestructura en dos zonas críticas, tal y como se ilustra detalladamente en el diagrama topológico de la Figura 4.2:

- **Subred Pública (Exposición a Internet):** En esta subred reside el único componente con una dirección IP pública estática accesible desde el exterior: un balanceador de carga (*Load Balancer*) flexible proporcionado por OCI, configurado para transmitir el tráfico en Capa 4. Al operar en la capa de transporte, este componente gestiona el tráfico a nivel de protocolo TCP y puertos, sin inspeccionar el contenido de los paquetes. Su función es actuar como la puerta principal del sistema, recibiendo todo el tráfico bruto en los puertos 80 (HTTP) y 443 (HTTPS) y redirigiéndolo de forma eficiente hacia el controlador de *Ingress* (Capa 7) que reside dentro del clúster. Para facilitar el acceso de los usuarios finales, se ha vinculado esta dirección IP pública con el dominio `spiritblade.dev`, el cual ha sido adquirido a través del registrador *name.com*. La gestión de los registros DNS en dicho proveedor permite mapear el dominio con la infraestructura de Oracle de forma transparente para el usuario.
- **Subredes Privadas (Aislamiento físico):** En estas subredes, ocultas detrás de un *NAT Gateway* (que permite la salida a Internet para descargar actualizaciones pero bloquea cualquier conexión entrante directa), se alojan los nodos de trabajo físicos (*Worker Nodes*) del clúster de Kubernetes y la conexión a la base de datos autónoma. Todo el código de la aplicación (los contenedores del cliente y el servidor), así como la caché interna de Redis, operan aquí en direcciones IP privadas (rango `10.0.10.0/24`). El tráfico entrante solo los alcanza tras ser validado y redirigido (*proxied*) por el *Ingress* desde la subred pública.

Para sortear las severas limitaciones de cómputo del *Free Tier* expuestas anterior-

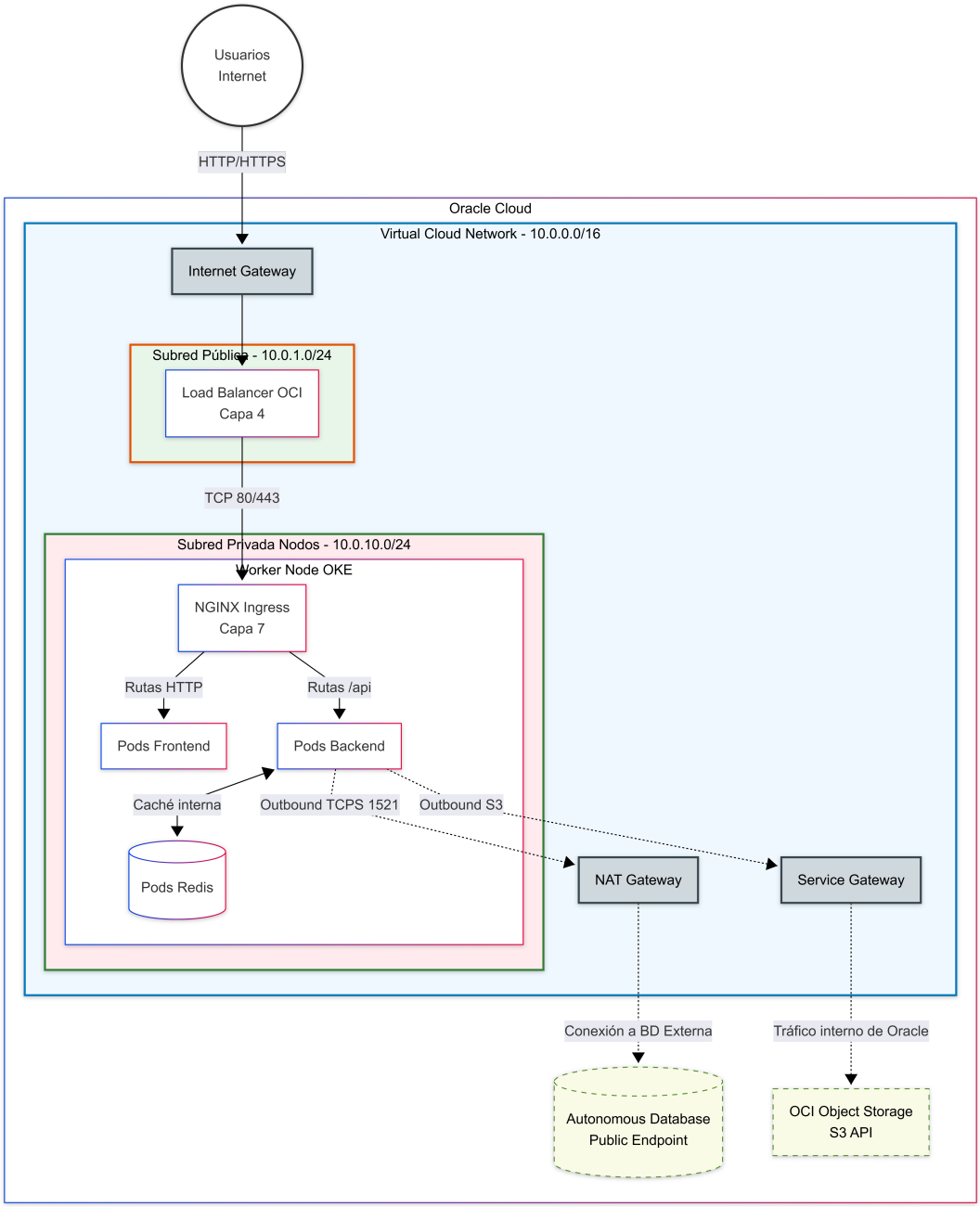


Figura 4.2: Diagrama topológico de la red virtual (*VCN*), sus subredes y puertas de enlace en Oracle Cloud.

mente, se externalizó la capa de persistencia hacia servicios gestionados que operan bajo sus propias cuotas gratuitas paralelas. Se provisionó una *Autonomous Database* (ADB) para los datos relacionales y un *OCI Object Storage* para los recursos multimedia. Al delegar estas tareas, ninguno de estos pesados servicios consume los valiosos núcleos *ARM* del clúster, lo que permitió dedicar el 100 % de las *OCPUs* permitidas exclusivamente a las máquinas virtuales de los nodos de *OKE*.

Al residir fuera de la red virtual (*VCN*), el acceso a estos componentes desde los nodos aislados en la subred privada requiere estrategias de enrutamiento distintas apoyadas en puertas de enlace (*Gateways*), tal y como refleja el diagrama 4.2:

- **Enlace hacia *Autonomous Database* (ADB):** Al verse forzado arquitectónicamente a desplegarse con un *endpoint* público en la capa *Always Free*, el tráfico desde los contenedores del servidor debe salir forzosamente a través del ***NAT Gateway***. Este componente actúa como proxy permitiendo la salida a la Internet pública mediante una conexión fuertemente cifrada (*TCPS*), bloqueando de forma inherente cualquier tráfico entrante no solicitado de vuelta hacia los nodos.

- **Enlace hacia *OCI Object Storage*:** Por su parte, al estar integrado nativamente en la red de servicios interna de Oracle (*Oracle Services Network*), las peticiones no necesitan exponerse a la Internet pública. Se ha configurado un ***Service Gateway*** que permite a los nodos comunicarse de forma privada, rápida y directa con las *APIs* de almacenamiento, manteniéndose todo el enrutamiento *HTTP/S3* dentro de la resguardada malla de seguridad del propio proveedor en la nube.

Dentro del ecosistema de Kubernetes, los contenedores originalmente orquestados con *Docker Compose* se tradujeron a múltiples recursos u objetos nativos:

- ***Deployments*:** Se definieron para el cliente web (*Angular*) y el servidor (*Spring Boot*). Estos manifiestos garantizan que siempre haya al menos una réplica del contenedor ejecutándose (*Pods*) y gestionan las actualizaciones de versión de forma controlada.

- ***Services*:** Se configuraron servicios internos de tipo *ClusterIP* para proporcionar descubrimiento de red y balanceo de carga interno entre los distintos *Pods* sin exponerlos a Internet.

- ***Ingress Controller*:** Se instaló un controlador *NGINX* acoplado a un gestor de certificados (*cert-manager*) para administrar el tráfico HTTP/HTTPS entrante y realizar la terminación segura (*SSL*) apoyado por *Let's Encrypt*.

- ***Helm Charts*:** Como se introdujo en apartados anteriores, la complejidad del despliegue de *Redis* (que requiere la orquestación simultánea de múltiples nodos primarios y réplicas) se resolvió inyectando el *chart* oficial de *Bitnami*.

El resultado final de esta provisión e interconexión de recursos se puede observar esquematizado en la Figura 4.3, que ilustra la visión general de la arquitectura del clúster.

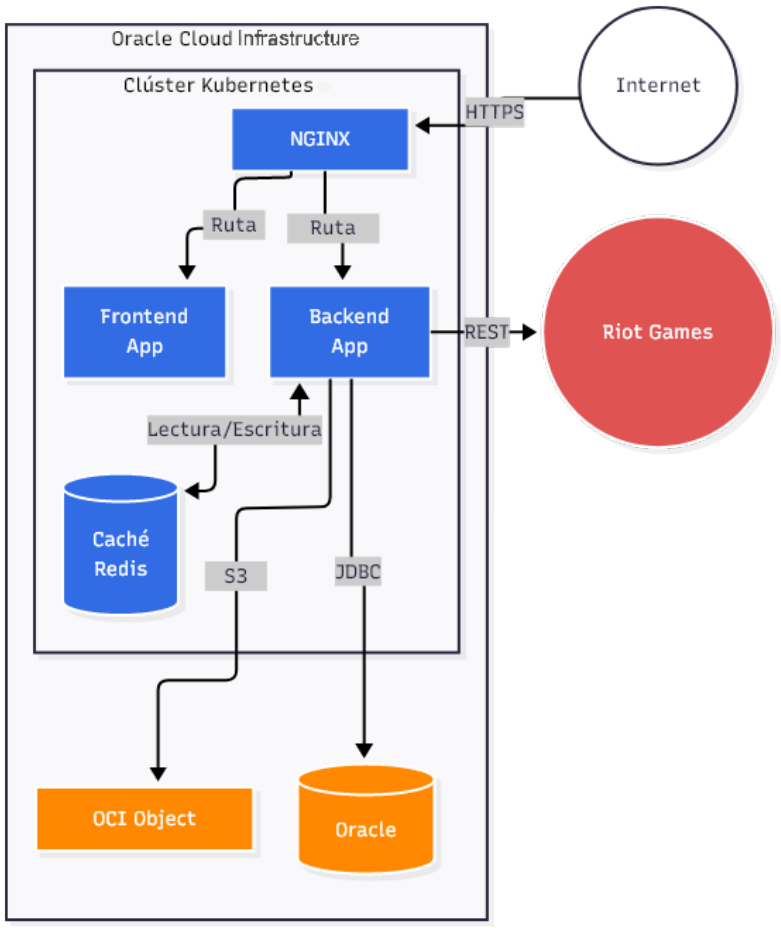


Figura 4.3: Visión general de alto nivel de los recursos del clúster de Kubernetes.

Problemas encontrados y soluciones

Durante las pruebas de despliegue inicial con *Terraform*, se afrontó una parálisis total del aprovisionamiento de la infraestructura, imposibilitando la creación de las máquinas virtuales. Este error no se debió a un fallo en el código, sino a la falta física de instancias *ARM* gratuitas en los centros de datos de la región asignada por *OCI* (error *Out of Capacity*). Ante el fracaso de los bucles automáticos de reclamación implementados, la solución operativa consistió en actualizar la cuenta *Always Free* al modelo *Pay As You Go* registrando un método de pago válido. Este simple cambio administrativo otorgó al

proyecto prioridad en la asignación de hardware dentro de la región de *Oracle*, permitiendo la creación inmediata del clúster físico. Cabe destacar que el proyecto se mantuvo sin coste económico alguno, ya que el consumo nunca excedió las cuotas estipuladas en la capa gratuita.

4.4.3. Fase 3: Escalabilidad

Objetivos de la fase

Dotar a la plataforma de elasticidad multinivel para soportar picos repentinos de tráfico concurrente sin degradación ni latencia, justificando minuciosamente las estrategias empleadas componente a componente bajo las estrictas restricciones de la capa *Always Free* de Oracle Cloud.

Detalles de implementación por componente

Como ya se ha comentado, los límites físicos de cómputo y almacenamiento impuestos por el *Free Tier* dictaminaron la filosofía de diseño. El escalado vertical (*Vertical Pod Autoscaler* o *VPA* del inglés) fue descartado de plano. Además de la limitación general del aumento exponencial de costes asociados a máquinas más grandes, en nuestro caso escalar verticalmente obligaba a reiniciar los servicios provocando caídas de disponibilidad (*downtime*), y resultaba inviable debido al tope de recursos general. Por ello, la arquitectura abrazó un modelo de **escalado horizontal**, distribuyendo la carga progresivamente en múltiples *Pods* efímeros.

- **Aplicación (*Frontend* y *Backend*):** Se implementó un *HPA* (*Horizontal Pod Autoscaler*) configurado para evaluar el consumo de *CPU*. Al no especificarse políticas de comportamiento personalizadas (*behaviors*) en los manifiestos *YAML*, *Kubernetes* aplica su algoritmo por defecto: el escalado es estrictamente progresivo y proporcional a la carga. El *HPA* calcula la cantidad de réplicas deseadas dividiendo el uso actual de CPU entre el objetivo (70 %) y multiplicándolo por las réplicas actuales. De este modo, escala de forma matemática y fluida de 1 hasta un máximo de 4 *Pods* en el cliente, y de 1 hasta 5 *Pods* en el servidor, instanciando únicamente los contenedores estrictamente necesarios para estabilizar la métrica. Se definieron cuotas estrictas de recursos para garantizar este balanceo: cada *Pod* del *frontend* tiene asignada una fracción de 100 milinúcleos (*100m* de *CPU*) y 64 MB de *RAM*, mientras que el *backend* en *Java* tiene reservados *250m* de *CPU* y 400 MB de *RAM*.

- **Infraestructura (*Cluster Autoscaler*):** Para soportar la creación de estos nuevos *Pods*, el clúster arranca con solo 2 nodos físicos encendidos (cada nodo cuenta con 1 *OCPU* y 6 GB de *RAM*) para ahorrar recursos. Cuando el *HPA* intenta crear réplicas pero se agota la capacidad total de los servidores activos, los nuevos contenedores quedan marcados en un estado *Pending*. Al detectar estos *Pods* huérfanos sin ubicar en el panel de control de Kubernetes, el *Cluster Autoscaler* se comunica automáticamente con la *API* de *Oracle* para aprovisionar dinámicamente hasta 2 nodos físicos *ARM* de la misma capacidad, alcanzando así el tope máximo permitido por la capa gratuita.

- **Caché en memoria (*Redis*):** La manera más sencilla de escalar una base de datos en memoria es verticalmente dándole más *RAM*. Dadas nuestras restricciones, esto no era posible. La solución fue transicionar hacia un clúster horizontalmente escalable instalando el *Helm Chart* oficial de *Bitnami* configurado en arquitectura primaria-réplica (*Primary-Replica*). En este modelo, un único nodo primario asume todas las operaciones de escritura para evitar conflictos, mientras que los datos se sincronizan hacia múltiples nodos réplica (de 2 a 4 *Pods*, cada uno con *150m* de *CPU* y 256 MB de *RAM*) diseñados exclusivamente para absorber el alto volumen de lecturas. Además, para evadir el límite de almacenamiento físico (*Block Volumes*) impuesto por *OCI* (el cual queda completamente absorbido por los discos de arranque de los servidores al alcanzar el tamaño máximo del clúster de cuatro nodos), el servicio *Redis* se forzó a operar empleando almacenamiento volátil temporal (`emptyDir`). Esto significa que *Redis* mapea su almacenamiento al propio disco efímero del nodo físico; si el nodo muere, la instantánea de caché se pierde, pero el sistema se autorrecupera poblándose de nuevo desde la base de datos principal al arrancar.

- **Base de datos relacional (*Autonomous Database*):** Se descartó albergar una base de datos *MySQL* tradicional dentro de *Kubernetes*, dado que habría devorado los recursos de cómputo del clúster. Al externalizarla hacia el servicio *ADB*, el escalado de conexiones concurrentes y el ajuste de poder de cómputo se delegan en el proveedor nube de manera invisible (*Serverless*), operando en su propia cuota independiente y sin consumir núcleos del clúster.

- **Almacenamiento de objetos (*S3*):** Los servicios *Object Storage* son inherentemente elásticos y *Cloud-Native*. Soportan incrementos masivos en la subida y lectura de imágenes o archivos sin configuraciones de *autoscaling* explícitas, actuando como un almacén infinito desvinculado de la capacidad de procesamiento de la *VCN*.

Para facilitar la comprensión visual de las estrategias de escalado horizontal adopta-

das en cada uno de los niveles de la arquitectura del clúster, se desglosan a continuación los diagramas de la Figura 4.4 (que ilustra el enrutamiento *Ingress* y escalado de *Pods* del cliente web), la Figura 4.5 (mostrando las réplicas del servidor *Spring Boot*) y la Figura 4.6 (ejemplificando la externalización hacia servicios gestionados y la arquitectura primaria-réplica de la caché *Redis* temporal).

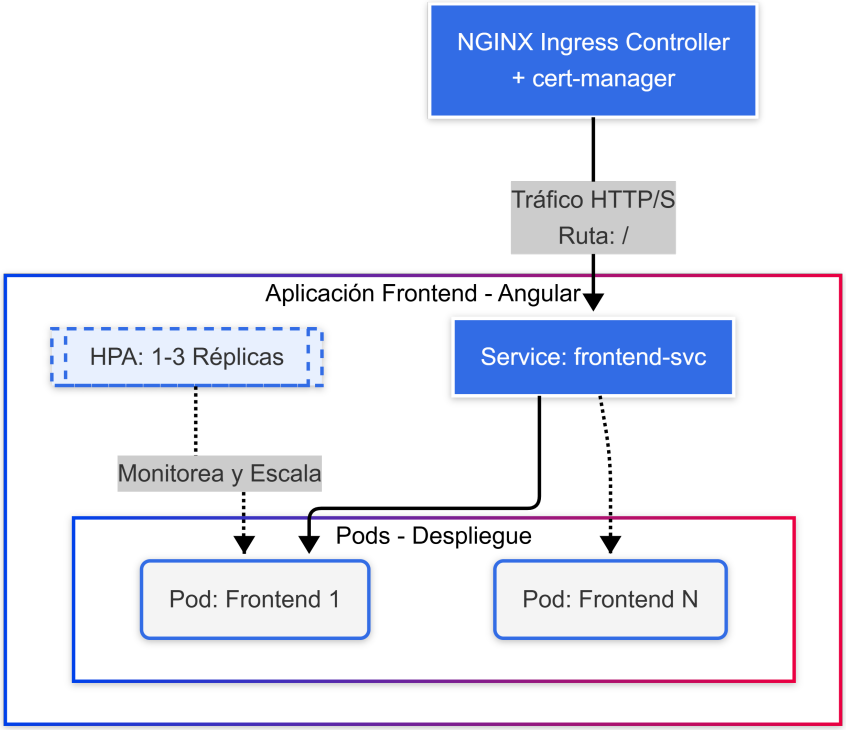


Figura 4.4: Topología y escalado horizontal de la capa de Frontend en Kubernetes.

Problemas encontrados y soluciones

Se detectó un bucle letal de autoescalado (*CrashLoop*) originado en el *backend*. Al intentar configurar el *HPA* basándose en consumo de *RAM*, la preasignación masiva de memoria (*heap*) inicial inherente a la *Java Virtual Machine* (*JVM*) emitía un falso positivo de saturación al clúster (detectando un 144 % de uso sobre la reserva base), el cual reaccionaba creando réplicas indiscriminadamente aunque no hubiese tráfico. La solución técnica fue retirar la monitorización *RAM* y aislar el *HPA* de *Java* exclusivamente al consumo de *CPU*, ya que el *Garbage Collector* de *Spring Boot* no devuelve la memoria *RAM* al sistema operativo inmediatamente, haciéndola una métrica ineficaz para medir carga real.

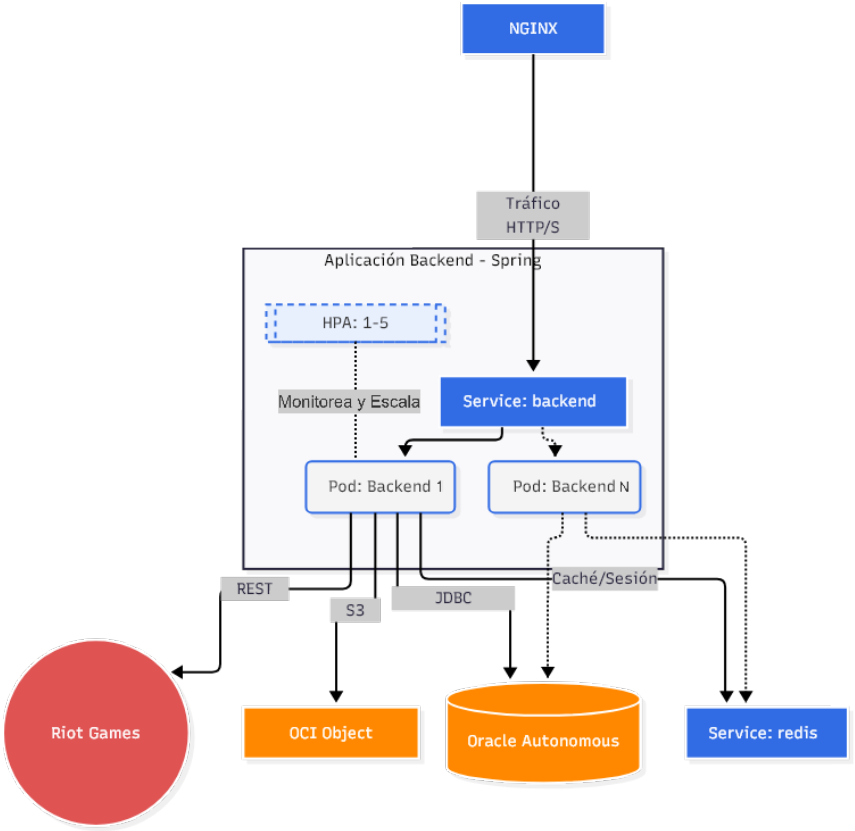


Figura 4.5: Topología y escalado horizontal de la capa de Backend en Kubernetes.

4.4.4. Fase 4: Despliegue continuo

Objetivos de la fase

Automatizar el ciclo de vida (*CI/CD*) para integrar código y publicarlo en el clúster sin intervención humana.

Detalles de implementación y problemas

La plataforma se dotó de un flujo completo en *GitHub Actions* que compila imágenes multi-arquitectura y actualiza los despliegues remotos. Para evitar redundancia, la descripción técnica profunda de estos flujos, la solución al cuello de botella de compilación *ARM/AMD64* y su arquitectura final se exponen detalladamente más adelante en la sección de *Proceso de desarrollo* (Sección 4.5).

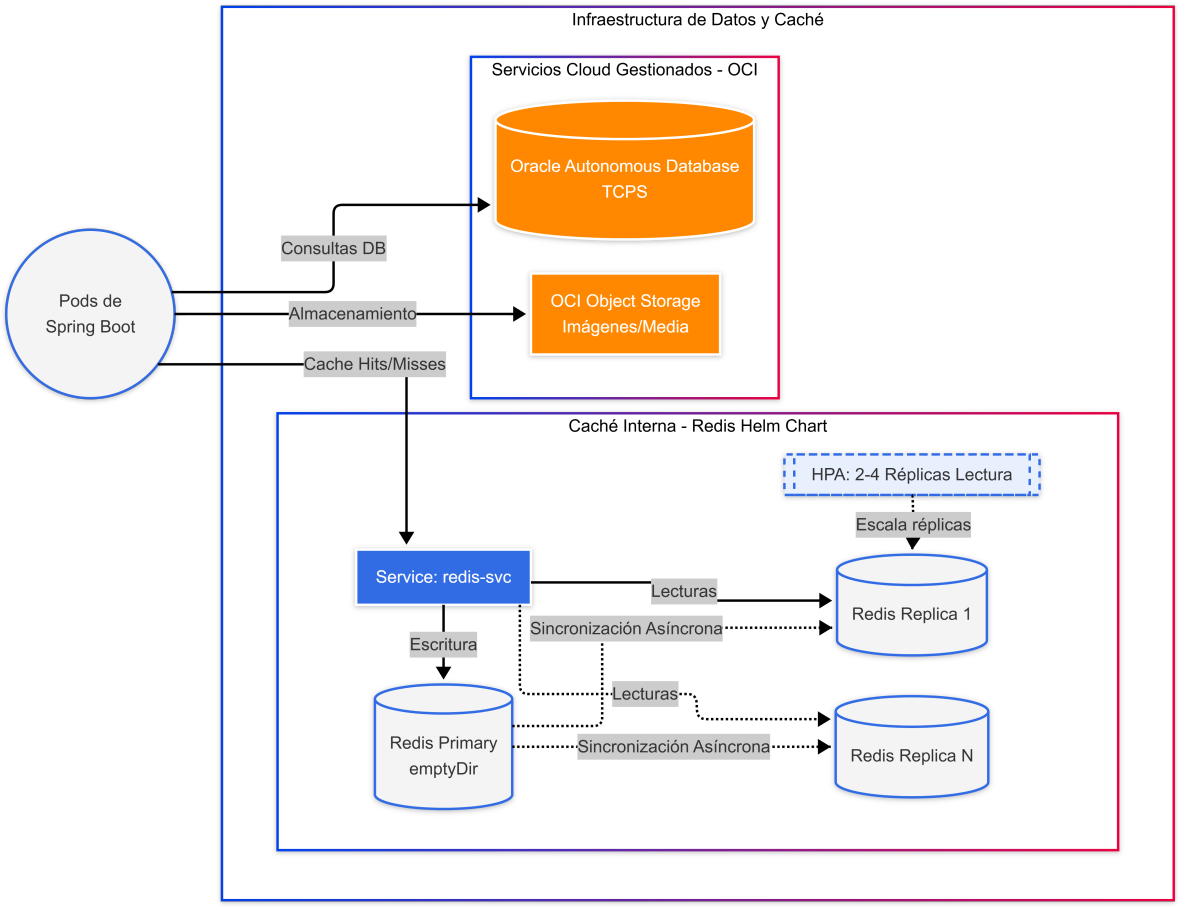


Figura 4.6: Arquitectura de la capa de Datos (Persistencia externa y Caché de lectura).

4.5. Proceso de desarrollo

El flujo de vida del código respeta las prácticas modernas de *GitOps* [8] apoyadas en tableros ágiles (*Kanban*), herramientas referenciadas como el estándar de la industria y que ya fueron introducidas y justificadas en el primer TFG de este proyecto. En la práctica, aplicar la filosofía *GitOps* en este proyecto significa establecer a *Git* (a través del repositorio de *GitHub*) como la única fuente de la verdad (*Single Source of Truth*). No solo el código fuente de la aplicación (*Angular* y *Spring Boot*) se encuentra versionado, sino que la infraestructura en la nube (*Terraform*) y el estado de los despliegues (*Kubernetes*) también residen en el mismo repositorio de forma declarativa. De este modo, cualquier modificación en los servidores o en la red no se realiza manualmente en la consola del proveedor *Cloud*, sino mediante alteraciones en los ficheros correspondientes, que desencadenan actualizaciones automáticas garantizando la trazabilidad y replicabilidad.

Para coordinar el desarrollo de forma metódica y mantener un historial rastreable, se ha mantenido la convención de nomenclatura establecida en la fase anterior. Cada característica de infraestructura o código se aísla en ramas semánticas siguiendo el formato `Feat(n)-Descripción`, donde "n" corresponde al identificador de la tarea en el panel *Kanban*. Esta misma nomenclatura se aplica de forma estricta a los mensajes de *commit*, garantizando una trazabilidad total entre los cambios en el código y los requisitos funcionales. En la Figura 4.7 se ilustra un ejemplo real de esta convención aplicada a una petición de integración (*Pull Request*), donde se aprecia claramente la coherencia semántica entre el nombre de la rama y los *commits* que la componen.

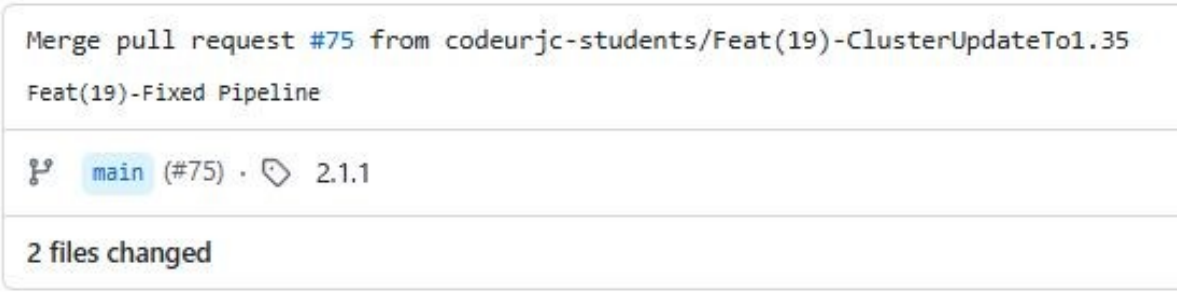


Figura 4.7: Ejemplo de petición de integración (*Pull Request*) mostrando la convención de nomenclatura aplicada tanto al nombre de la rama superior como a los mensajes de los *commits* individuales subyacentes.

El empaquetado de la solución final se apoya en contenedores; el cliente y el servidor se empaquetan por separado en imágenes *Docker* independientes basadas en *Alpine Linux* (para minimizar la superficie de ataque y el tamaño final), las cuales se distribuyen mediante el registro oficial *Docker Hub*.

4.5.1. Integración, Entrega y Despliegue Continuo (*CI/CD*)

Aunque el anterior TFG sentó las bases de la Integración Continua (compilación y análisis estático), en esta segunda iteración dichos flujos se han rediseñado por completo y expandido para abarcar las exigencias de un clúster *Cloud-Native* real. El ciclo de *GitHub Actions* actúa como el corazón metodológico y operativo del proyecto. Para conseguir una entrega y despliegue continuo fluidos, se han segmentado los procesos en distintos *workflows* especializados. La interconexión y ejecución secuencial de las fases de construcción y validación se ilustra en la Figura 4.9:

- **Integración Continua (`build.yml`):** Se encarga del control de calidad del código. Define una estrategia dual: en ramas de características (*feature branches*) se ejecutan análisis básicos y pruebas unitarias aisladas. Cuando se realiza un *Pull Request* hacia la rama principal, se levantan servicios efímeros reales de *MySQL*, *Redis* y *MinIO* mediante contenedores durante la ejecución de los *workflows*. Esto permite validar mediante nuevos casos de prueba de integración que la lógica de negocio interactúa correctamente con la caché de *Redis* y el *Object Storage* antes de cada despliegue. Finalmente, se inyectan las métricas en *SonarCloud* (configurado en el TFG previo) para evaluar la calidad estática del código, superando todas las compuertas de calidad (*Quality Gates*) sin inyectar vulnerabilidades, tal como se muestra en la Figura 4.8.
- **Construcción de Imágenes (`build-push.yml`):** Actúa como un flujo reutilizable que construye simultáneamente las imágenes *Docker* y los *artifacts* de la aplicación.
- **Entrega y Despliegue Continuo (`deploy-dev.yml` y `deploy-release.yml`):** Automatizan la puesta en producción sin intervención humana hacia dos entornos separados. El primero despliega en un *namespace* de desarrollo (`dev`) para pruebas internas tras cada integración en la rama principal. El segundo actúa como el mecanismo de lanzamiento a producción real (`prod`): reacciona automáticamente ante el etiquetado de nuevas versiones en el repositorio (*tags* semánticos tipo `v1.1.0`), conectándose a *OCI* e inyectando las nuevas versiones en el clúster, tal como se detalla posteriormente.

El proceso de Despliegue Continuo (*CD*) en *Kubernetes*

La automatización del despliegue en la nube requiere un manejo estricto de credenciales e interacciones con el clúster remoto. El *pipeline* de lanzamiento a producción (`deploy-release.yml`) se divide en dos trabajos (*jobs*), representados gráficamente en la Figura 4.10:

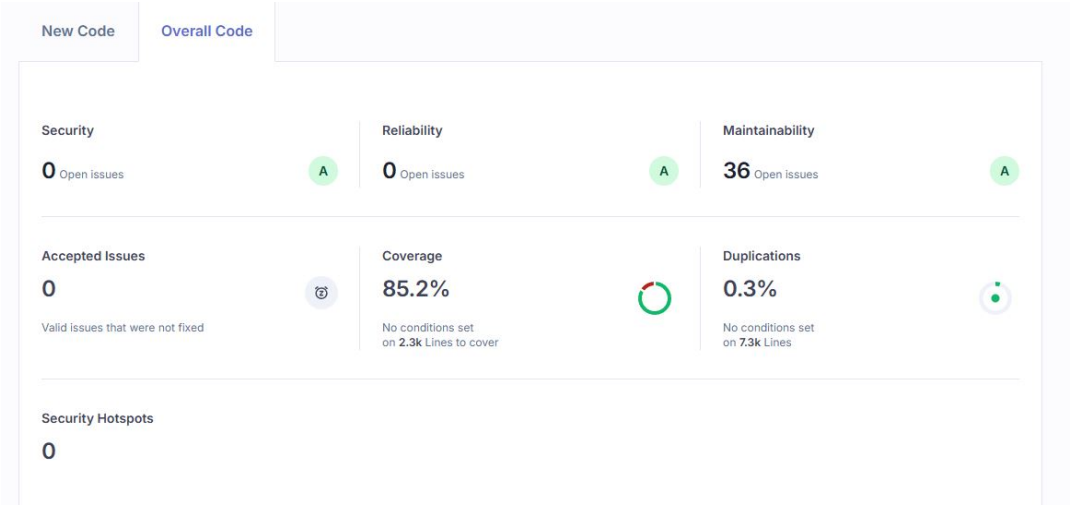


Figura 4.8: Captura de pantalla de los resultados del análisis estático de código en *SonarCloud*.

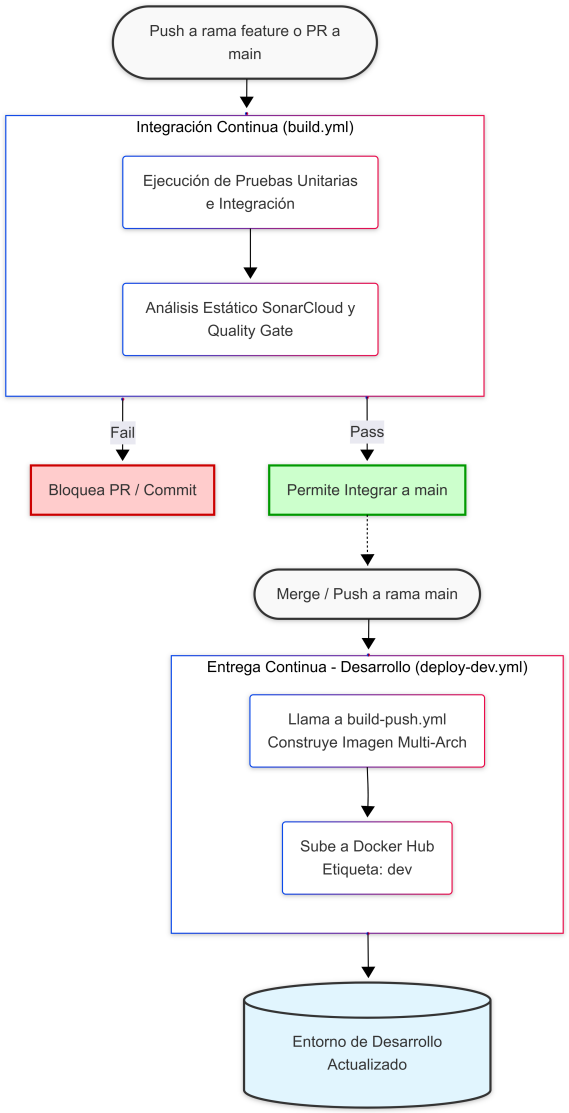


Figura 4.9: Diagrama de flujo de Integración Continua y validación de código.

1. **Construcción y Registro:** Invoca el flujo reutilizable para publicar las imágenes etiquetadas.

2. **Intervención en el Clúster:** Utilizando credenciales inyectadas de forma segura, el sistema se autentica en la cuenta de *Oracle Cloud*, obtiene acceso a *Kubernetes* y ejecuta una actualización ordenando a los *Deployments* del cliente y servidor que destruyan las réplicas antiguas antes de desplegar las nuevas (estrategia *Recreate*). Antes de la versión 1.35 de OKE, el clúster intentaba realizar despliegues sin interrupción (*Rolling Updates*), levantando los *Pods* nuevos antes de apagar los antiguos; sin embargo, esta estrategia duplicaba momentáneamente el consumo de *RAM* y *CPU*, provocando bloqueos fatales al chocar directamente contra los límites físicos (24 GB de *RAM*) de la capa gratuita, por lo que se abandonó el *zero-downtime* para asegurar la supervivencia de la infraestructura.

Cuello de botella de arquitectura mixta en la compilación *Docker*

Durante la confección del *Pipeline* de *CI/CD*, se originó un conflicto severo. Los nodos de la nube requerían imágenes nativas para *ARM64*, mientras que los servidores de *GitHub Actions* operaban sobre *AMD64*. Inicialmente, al emular el juego de instrucciones *ARM* mediante *QEMU* dentro de *Docker* para compilar el código fuente en *Java* y *Angular*, se generó un cuello de botella que disparó los tiempos de ejecución y causó *timeouts* catastróficos. La solución fue extraer la compilación de código fuera de *Docker*, forzándola a ejecutarse nativamente en corredores asignados de *GitHub* (Código 4.2). Consecuentemente, los `Dockerfile` (Código 4.3) se redujeron a ser envolturas que simplemente empaquetan el *artifact* pre-compilado. Este ajuste redujo los tiempos de construcción de horas a escasos minutos.

Código 4.2: Fragmento de GitHub Actions ejecutando compilación nativa en matriz.

```
jobs:
  build:
    strategy:
      matrix:
        include:
          - arch: amd64
            runner: ubuntu-latest
          - arch: arm64
            runner: ubuntu-24.04-arm
    runs-on: ${{ matrix.runner }}
    steps:
      - name: Compilar codigo nativamente
        run: mvn clean package -DskipTests
```

Código 4.3: Dockerfile de backend optimizado (sin compilación interna).

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY target/*.jar app.jar
```

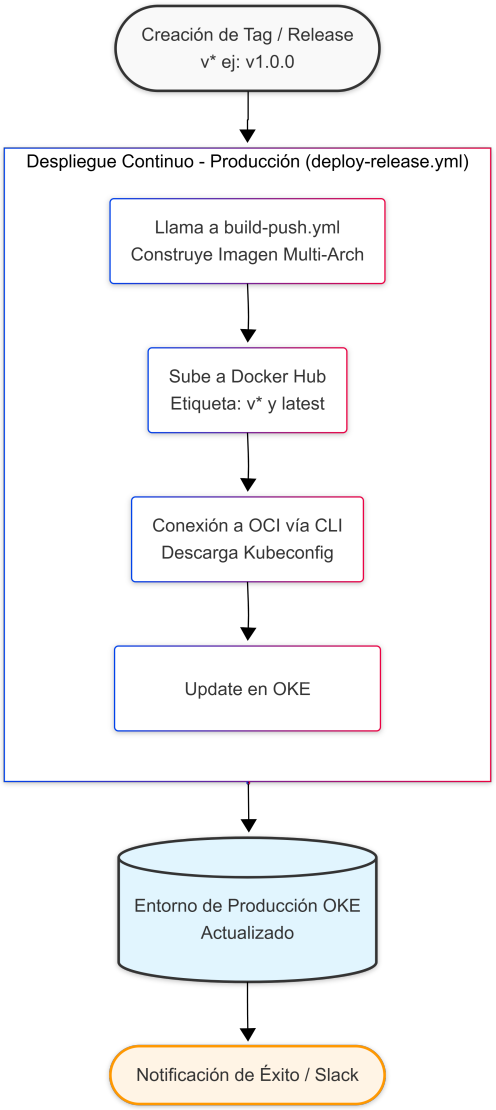


Figura 4.10: Diagrama de flujo de Despliegue Continuo a Producción mediante versionado.

```
4  EXPOSE 8080
5  ENTRYPOINT ["java", "-jar", "app.jar"]
```

4.6. Pruebas de carga y validación de escalabilidad

Para garantizar la fiabilidad del despliegue en la nube como respuesta a los mecanismos de autoescalado implementados en la arquitectura, se redactaron escenarios de simulación de tráfico con *Artillery* que se pueden encontrar completos en el repositorio público (`tests/load/artillery-config.yaml`). Estos *tests* se diseñaron para someter al clúster a una carga progresiva intensa.

Durante una prueba ininterrumpida de **7 minutos y 9 segundos**, el sistema absorbió un total de **66.523 peticiones *HTTP*** generando un pico sostenido de carga. La salida detallada y extensa se puede consultar en el repositorio, pero el resumen de los resultados métricos más relevantes se presenta en la Tabla 4.1.

Métrica de Rendimiento	**Resultado**
Total de usuarios virtuales creados	7.150
Total de peticiones HTTP procesadas	66.523
Tasa media de peticiones por segundo (RPS)	50 RPS (Picos de 56 RPS)
Tiempo de respuesta mediano (p50)	47 ms
Tiempo de respuesta percentil 95 (p95)	632,8 ms
Errores HTTP transitorios (502/503)	1,2 %

Tabla 4.1: Resultados globales de la prueba de carga con Artillery.

Estos resultados validan en tiempo real los ejes de escalado planificados: la avalancha de peticiones empujó el consumo de CPU por encima del 70 %, disparando el *HPA* para replicar los contenedores. Al llenarse los nodos, el *Cluster Autoscaler* actuó provisionando servidores físicos adicionales de manera dinámica.

Para ilustrar empíricamente este comportamiento bajo estrés, se monitorizó de forma ininterrumpida la respuesta de la infraestructura frente a los picos de carga. En la Figura 4.11 se presenta la evolución temporal del estado del clúster, donde se observa claramente cómo reacciona tanto el *backend* como el *frontend* frente a la saturación.

Como se aprecia en el gráfico, en el momento en que se intensifica el flujo de peticiones, la métrica de uso de CPU del *backend* (línea roja) experimenta un crecimiento drástico, superando rápidamente su límite configurado. Como respuesta directa, su controlador *HPA* ordena la instanciación de nuevas réplicas (escalonado azul), llegando a saturar la capacidad de los nodos existentes. Ante la falta de recursos físicos en el clúster para acomodar los nuevos *Pods*, el *Cluster Autoscaler* interviene y demanda el

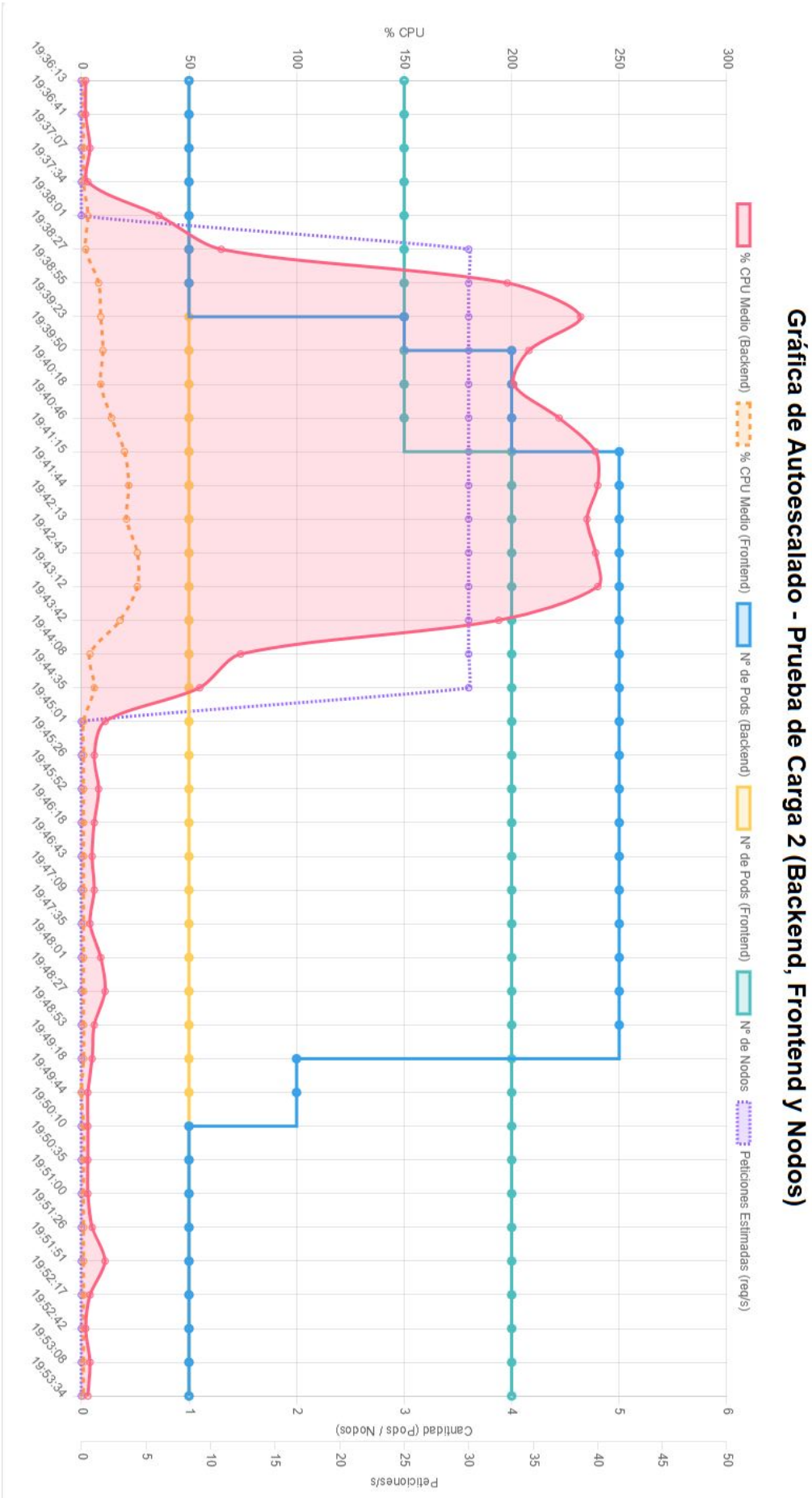


Figura 4.11: Evolución del autoescalado de *Pods* y Nodos en relación al porcentaje de uso de CPU durante la simulación de carga.

aprovisionamiento de un nuevo servidor al proveedor *Cloud* (escalonado turquesa).

Por el contrario, debido a su naturaleza estática y menor exigencia de cómputo, el *frontend* (línea naranja) apenas sufre variaciones en su consumo de CPU, manteniéndose muy por debajo de sus umbrales críticos. En consecuencia, el sistema mantiene de manera autónoma una única réplica activa para esta capa (línea amarilla). Este comportamiento demuestra la independencia y eficiencia del autoescalado, destinando los recursos físicos exclusivamente a los cuellos de botella reales de la aplicación.

5

Conclusiones y trabajos futuros

5.1. Conclusiones

La consecución de este TFG representa la culminación del ciclo de vida del *software* moderno. Mientras que el primer TFG se centró en la ideación y construcción funcional de la plataforma *SPIRITBLADE*, este segundo esfuerzo ha permitido elevar el sistema desde un entorno de desarrollo aislado hasta una infraestructura de producción real, distribuida, segura y resiliente. A continuación, se reflexiona sobre el cumplimiento de los objetivos establecidos al inicio del proyecto.

5.1.1. Reflexión sobre los objetivos funcionales

Desde la perspectiva funcional y de servicio al usuario, el objetivo principal de ofrecer una disponibilidad ininterrumpida se ha cumplido satisfactoriamente. La aplicación es accesible públicamente a través del dominio `spiritblade.dev`. Además, las comunicaciones están securizadas de manera transparente para el usuario mediante cifrado HTTPS, gestionado íntegramente por *cert-manager* y *Let's Encrypt*.

En cuanto a la tolerancia a fallos y escalabilidad, el sistema ha demostrado ser capaz de auto-repararse y ajustar sus recursos dinámicamente frente a simulaciones de carga,

garantizando que los picos de usuarios no degradan el rendimiento general del servicio ni la experiencia de análisis estadístico de partidas de *League of Legends*. Resulta especialmente destacable haber logrado todos estos hitos operando dentro de los límites y cuotas del *Free Tier* de Oracle Cloud, demostrando que es posible montar infraestructuras avanzadas minimizando costes si se optimizan exhaustivamente los componentes.

No obstante, este proceso de despliegue real ha destapado una de las lecciones más valiosas del proyecto: la gestión de expectativas y dependencias de proveedores externos (proveedores de nube). Al inicio del trabajo no se contempló que la capa 100 % gratuita vendría supeditada a una severa falta de disponibilidad de *hardware* (el problema de *Out of Capacity*). La necesidad de dejar un *script* programado ejecutándose ininterrumpidamente durante semanas en vano para poder acceder a los procesadores prometidos, ilustra de manera vívida la cruda realidad del mercado *Cloud*. La transición obligatoria hacia el modelo *Pay As You Go* para priorizar la cuenta y evitar las colas infinitas resalta que, incluso cuando los costes directos de cómputo pueden ser gratuitos (respetando los límites de uso), la dependencia comercial del servicio, las caídas del sistema o la gestión de cuotas y colas son fricciones inherentes al despliegue de *software* moderno a gran escala.

5.1.2. Reflexión sobre los objetivos técnicos

Técnicamente, el proyecto ha forzado la superación de paradigmas clásicos de desarrollo. La transición hacia contenedores (*Docker*) y orquestación masiva (*Kubernetes*) ha sido exitosa. La adopción del modelo de *IaC* mediante *Terraform* garantiza que el clúster puede ser destruido y levantado desde cero en cuestión de minutos, eliminando el error humano derivado de configuraciones manuales.

Asimismo, la resolución de incidencias arquitectónicas profundas, como el cuello de botella generado por la emulación de *QEMU* al intentar compilar código para arquitecturas *ARM* en procesadores tradicionales, o el falso positivo de los autoescaladores de la *Java Virtual Machine* (*JVM* del inglés) por reservas excesivas de memoria, subrayan el alto grado de madurez técnica alcanzado a lo largo del proceso. La implementación de un flujo *CI/CD* en *GitHub Actions* fluido, que compila artefactos multi-arquitectura en tiempo récord y evalúa estáticamente el código en *SonarCloud*, cierra el círculo de una verdadera filosofía *DevOps*.

5.1.3. Conclusiones personales

A nivel personal y académico, la realización conjunta de ambas memorias ha supuesto el reto más ambicioso de mi etapa universitaria. Si el primer TFG me permitió afianzar mis conocimientos como ingeniero de *software Full-Stack*, este segundo proyecto me ha obligado a adoptar la mentalidad de un ingeniero *Cloud* y *SRE*. He aprendido a no dar por sentado el entorno en el que corre el código, asimilando que la red falla, los nodos se caen y el tráfico es impredecible. La capacidad de diseñar sistemas que sobrevivan y prosperen bajo estas premisas ha expandido drásticamente mi visión integral del ciclo de vida del *software*.

5.2. Trabajos futuros

A pesar del sólido estado actual de la infraestructura, la evolución de los sistemas *Cloud-Native* es continua. Se proponen las siguientes líneas de trabajo para futuras iteraciones de la plataforma *SPIRITBLADE*:

- **Monitorización avanzada y Observabilidad:** Aunque *Kubernetes* proporciona el *Metrics Server* para el autoescalado, la visibilidad operativa mejoraría sustancialmente integrando un *stack* de observabilidad completo (ej. *Prometheus* y *Grafana*) para recopilar y visualizar métricas de negocio y rendimiento en tiempo real.
- **Seguridad y Protección Perimetral:** Para fortalecer la resiliencia del sistema ante ataques externos, una línea futura prioritaria es la implementación de protección avanzada contra denegación de servicio (*Distributed Denial of Service* o *DDoS* del inglés) y la integración de un *Web Application Firewall* (*WAF* del inglés) que filtre peticiones maliciosas antes de que alcancen el clúster.
- **Despliegue Multi-Región y Multi-Cloud:** Para mitigar el impacto de una caída completa de un centro de datos, se podría explorar la federación de clústeres en diferentes regiones geográficas de Oracle Cloud o incluso en otros proveedores de nube, garantizando una alta disponibilidad global.
- **Optimización de Base de Datos:** Evolucionar el modelo de persistencia actual hacia patrones de acceso más eficientes, evaluando el uso de bases de datos *Not Only SQL* (*NoSQL* del inglés) para el almacenamiento de historiales masivos de partidas que no requieren una estructura relacional estricta.

Bibliografía

[1] Oracle, "Oracle cloud infrastructure (oci)," https://www.oracle.com/cloud/, 2024.

[2] D. Inc., "Docker: Accelerated container application development," https://www.docker.com/, 2024.

[3] T. K. Authors, "Kubernetes," https://kubernetes.io/, 2024.

[4] T. H. Authors, "Helm: The package manager for kubernetes," https://helm.sh/, 2024.

[5] Oracle, "Oci command line interface (cli)," https://docs.oracle.com/en-us/iaas/Content/API/Concepts/cliconcepts.htm, 2024.

[6] T. K. Authors, "kubectl reference documentation," https://kubernetes.io/docs/reference/kubectl/, 2024.

[7] Artillery.io, "Artillery: Cloud-scale load testing," https://www.artillery.io/, 2024.

[8] R. Hat, "Guide to gitops," https://www.redhat.com/es/topics/devops/what-is-gitops, 2024.

Anexos

A

Instrucciones de Despliegue

Este anexo detalla los pasos técnicos y empíricos necesarios para desplegar la infraestructura y la aplicación *SPIRITBLADE* en *OCI*. A diferencia del entorno local, este despliegue materializa una arquitectura elástica y resiliente utilizando *Terraform* y *Kubernetes*.

A.1. Requisitos previos

Para la orquestación inicial del entorno desde una máquina local, se requiere:

- ***Terraform*:** Motor de *IaC* para el aprovisionamiento de recursos.
- ***OCI CLI*:** Configurada con las claves API del usuario administrador.
- ***kubectl*** y ***Helm*:** Herramientas de control del clúster *Kubernetes*.

A.2. Fase 1: Provisión de Infraestructura (*Terraform*)

Los archivos de configuración en `iac/terraform` definen la red (*VCN*), la base de datos autónoma (*ADB*) y el clúster gestionado (*OKE*).

1. **Configuración:** Copiar el ejemplo de variables y asignar los identificadores de Oracle:

```
cd iac/terraform
cp terraform.tfvars.example terraform.tfvars
# (Editar terraform.tfvars rellenando con los OCID de la cuenta de OCI)
```

2. **Despliegue de Recursos:** Debido a la alta demanda de instancias gratuitas ARM (*Out of Capacity*), el proyecto incluye un script de reintento automático (`conectTerraform.ps1`) para asegurar la adquisición de nodos físicos:

```
terraform init
./conectTerraform.ps1
```

3. **Conexión al Clúster y Variables Resultantes:** Una vez levantada la infraestructura, se vincula el terminal local con el clúster y se revisan las salidas (outputs) para la siguiente fase:

```
oci ce cluster create-kubeconfig --cluster-id <OCID> --file ~/.kube/config
terraform output
```

A.3. Fase 2: Configuración del Entorno *Kubernetes*

Con el clúster activo, se asocia la arquitectura de contenedores a los recursos físicos aprovisionados, inyectando los identificadores de Terraform en los manifiestos de la carpeta `k8s/prod`:

1. **Preparación del Clúster y Enlaces:** Crear el espacio de nombres de producción y poblar los manifiestos con los valores dinámicos resultantes de Terraform:

```
kubectl apply -f k8s/prod/namespace.yaml
# Sustituir 'ocid1.nodepool...' por 'nodepool_id' en
    k8s/prod/cluster-autoscaler.yaml
# Sustituir la URL JDBC por 'adb_jdbc_url' en k8s/prod/backend-deployment.yaml
# Sustituir el EndPoint S3 por 's3_compatible_endpoint' en
    backend-deployment.yaml
```

2. **Cifrado de Credenciales:** Configurar las credenciales (copiando el archivo de ejemplo) y aplicarlas:

```
cp k8s/prod/secrets.yaml.example k8s/prod/secrets.yaml
# (Editar k8s/prod/secrets.yaml con las credenciales en Base64)
kubectl apply -f k8s/prod/secrets.yaml
```

3. **Controladores Base:** Instalar NGINX Ingress Controller y Cert-Manager (certificados SSL):

```
# NGINX Ingress Controller
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ \
  ingress-nginx/controller-v1.10.0/deploy/static/provider/cloud/deploy.yaml

# Cert-Manager
kubectl apply -f https://github.com/cert-manager/cert-manager/ \
  releases/download/v1.14.2/cert-manager.yaml
```

4. **Infraestructura Complementaria:** Añadir el repositorio de Bitnami e instalar la caché *Redis* mediante *Helm* para optimizar las consultas externas:

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm install redis bitnami/redis -f k8s/prod/redis-values.yaml -n prod
```

5. **Despliegue de la Aplicación:** Aplicar de forma masiva los despliegues del cliente, servidor, recolector de métricas, rutas de red (*Ingress*) y políticas de escalado (*HPA*):

```
kubectl apply -f k8s/prod/
```

A.4. Fase 3: Entrega Continua (*CI/CD*)

Tras la configuración inicial, las futuras actualizaciones de la aplicación no requieren intervención manual mediante terminal. La infraestructura está acoplada al repositorio remoto a través de ***GitHub Actions***.

Para lanzar una nueva versión de la aplicación sin cortes de servicio:

1. **Configuración de Secretos en GitHub:** Asegurarse de tener configurados los secretos requeridos en los ajustes del repositorio: `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN`, `OCI_CLI_USER`, `OCI_CLI_FINGERPRINT`, `OCI_CLI_TENANCY` y `OCI_CLI_KEY_CONTENT`.
2. Crear una nueva versión semántica en el repositorio remoto (*Tag*):

```
git tag v1.2.0
git push origin v1.2.0
```

3. La acción automatizada `deploy-release.yml` detectará la nueva etiqueta, compilará nativamente las imágenes para arquitectura ARM y AMD64, las inyectará en *Docker Hub* y enviará la orden de actualización continua (*Rolling Update*) al clúster de producción para reflejar los cambios globalmente.

A.5. Fase 4: Configuración de Dominio y DNS

Para finalizar el despliegue y permitir el acceso a través del dominio `spiritblade.dev`, se deben configurar los registros en el proveedor de DNS (*name.com*):

1. **Obtener IP Pública:** Identificar la IP estática asignada al balanceador de carga en la consola de OCI o mediante `kubectl`:

```
kubectl get svc ingress-nginx-controller -n ingress-nginx
```

2. **Configurar registros A:** En el panel de control de *name.com*, añadir los siguientes registros de tipo A:
   - **Host:** `@` (o dejar vacío) — **Valor:** `<IP_PUBLICA>`
   - **Host:** `www` — **Valor:** `<IP_PUBLICA>`
3. **Verificación:** Comprobar la propagación del DNS y la correcta resolución del nombre:

```
nslookup spiritblade.dev
```